

Presented by

Data+

VALORANT PRO MATCHES

Felix Tjahyadi - 2106638614

Muhammad Hafizha Dhiyaulhaq - 2106750723

Lyzander Marciano Andrylie - 2106750755

VPM - DATA+



OUTLINE

1. INTRODUCTION

2. DATASET

3. INITIAL DATA PREPROCESSING

4. EXPLORATORY DATA ANALYSIS

5. DATA PREPROCESSING

6. CLUSTER ANALYSIS

7. MODELING AND EVALUATION



VALORANT

INTRODUCTION

- 5v5 team-based first person shooter dari Riot Games.
- Pemain akan menggunakan agent dengan kemampuan unik dengan peran unik: duelists, sentinels, initiators, atau controllers.
- Memiliki kancah esports aktif dengan hadiah kemenangan yang besar.

PRIZE POOL 2023 VALORANT CHAMPIONS

\$2.25 MILLION



TUJUAN

- Menganalisis data pertandingan dari **Valorant Pro Matches**
- Memprediksi **Agent** yang digunakan oleh setiap **pemain** pada setiap **babak**.
- Memprediksi nilai rata-rata **ACS** dari masing-masing tim untuk sebuah pertandingan.
- Clustering pada dataset, analisis, dan insight dari cluster yang dibuat.

STAKEHOLDER

- Pelatih Tim Esport Valorant
- Valorant Esport Analyst
- Valorant Esport Broadcaster
- Pemain Profesional Valorant

DATASET

Pada topik valorant pro matches, terdapat tiga dataset yaitu matches, games, dan scores yang masing-masing dataset berisikan informasi mengenai statistik pertandingan pro player pada sebuah turnamen.

DATA SET MATCHES

Berisi informasi tentang tim yang berpartisipasi dalam sebuah turnamen. Pertandingan umumnya diadakan dengan aturan "best of 3," di mana tim yang memenangkan babak terbanyak akan menjadi pemenang pertandingan.



INFORMASI FITUR DATASET MATCHES

DATASET MATCHES

Nama Fitur	Deskripsi
MatchID	Id pertandingan
Date	Tanggal pertandingan
Patch	Versi Valorant saat pertandingan dilakukan
EventID	Id turnamen
EventName	Nama turnamen
EventStage	Tahap turnamen

INFORMASI FITUR DATASET MATCHES

DATASET MATCHES

Nama Fitur	Deskripsi
Team1ID	Id tim 1
Team2ID	Id tim 2
Team1	Nama tim 1
Team2	Nama tim 2
Team1_MapScore	Banyak babak yang dimenangkan tim 1
Team2_MapScore	Banyak babak yang dimenangkan tim 2

DATASET GAMES

Mendokumentasikan detail babak yang dimainkan oleh kedua tim yang bersaing dalam turnamen.



INFORMASI FITUR DATASET GAMES

DATASET GAMES

Nama Fitur	Deskripsi
GameID	Id babak
MatchID	Id pertandingan
Map	Peta yang digunakan pada sebuah babak
Team1ID, Team2ID	Id tim 1 dan tim 2
Team1, Team2	Nama tim 1 dan tim 2
Winner	Tim yang menjadi pemenang pada sebuah babak

INFORMASI FITUR DATASET GAMES

DATASET GAMES

Nama Fitur	Deskripsi
Team1_Eco, Team2_Eco	Banyak ronde eco yang dilakukan tim 1 dan tim 2
Team1_SemiEco, Team2_SemiEco	Banyak ronde semi eco yang dilakukan tim 1 dan tim 2
Team1_SemiBuy, Team2_SemiBuy	Banyak ronde semi buy yang dilakukan tim 1 dan tim 2
Team1_FullBuy, Team2_FullBuy	Banyak ronde full buy yang dilakukan tim 1 dan tim 2
Team1_TotalRounds, Team2_TotalRounds	Banyak ronde yang dimenangkan tim 1 dan tim 2

DATASET SCORES

Merupakan catatan akhir dari status para pemain profesional yang bertanding dalam babak tersebut



INFORMASI FITUR DATASET SCORES

DATASET SCORES

Nama Fitur	Deskripsi
GameID	Id babak
PlayerID	Id pemain
PlayerName	Username pemain
TeamAbbreviation	Singkatan nama tim
Agent	Agent/karakter yang digunakan
ACS	Skor rata-rata dari pemain
Kills	Jumlah musuh yang dikalahkan

INFORMASI FITUR DATASET SCORES

DATASET SCORES

Nama Fitur	Deskripsi
Deaths	Total pemain tereliminasi
Assists	Jumlah kontribusi pemain dalam membantu rekannya untuk mengalahkan musuh
PlusMinus	Selisih antara jumlah mengalahkan musuh (Kills) dan jumlah tereliminasi (Deaths)
KAST_Percent	Besar kontribusi pemain dalam pertandingan
ADR	Rata-rata serangan yang diberikan oleh pemain ke pihak musuh
HS_Percent	Persentase serangan yang diberikan mengenai kepala musuh
FirstKills	: Total pemain berhasil melakukan eliminasi pertama kali pada suatu ronde

INFORMASI FITUR DATASET SCORES

DATASET SCORES

Nama Fitur	Deskripsi
FirstDeaths	Total pemain dieliminasi pertama kali pada suatu ronde
FKFD_PlusMinus	Selisih antara jumlah First Kills dan First Deaths
Num_2Ks, Num_3Ks, Num_4Ks, Num_5Ks	Jumlah pemain berhasil mengeliminasi X musuh dalam suatu ronde
OnevOne, OnevTwo, OnevThree, OnevFour, OnevFive	Jumlah pemain berhasil mengalahkan X musuh pada suatu timeframe
Econ	Besar serangan yang dihasilkan untuk setiap 1000 creds (mata uang pada permainan) yang digunakan untuk membeli perlengkapan
Plants	Jumlah pemain berhasil menanam spike (bom) pada suatu ronde
Defuses	Jumlah pemain berhasil menjinakkan spike (bom) pada suatu ronde

INITIAL DATA PREPROCESSING

DATASET MATCHES

```

RangeIndex: 6401 entries, 0 to 6400
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   No                   6401 non-null   int64
1   MatchID              6401 non-null   int64
2   Date                 6401 non-null   object
3   Patch                5990 non-null   object
4   EventID              6401 non-null   int64
5   EventName            6401 non-null   object
6   EventStage           6401 non-null   object
7   Team1ID              6401 non-null   int64
8   Team2ID              6401 non-null   int64
9   Team1                6401 non-null   object
10  Team2                6401 non-null   object
11  Team1_MapScore        6401 non-null   int64
12  Team2_MapScore        6401 non-null   int64
dtypes: int64(7), object(6)

```

Imputasi Kolom PlayerID

Memberikan ID unik pada pemain dengan PlayerName tertentu

Transformasi Kolom Date

Menghapus waktu pada Date.

Contoh: "2022-01-08 15:30:00" menjadi "2022-01-08"

Transformasi dan Imputasi Kolom Patch

Patch	Date	
	min	max
0	2020-05-04	2020-06-14
1	2020-06-04	2021-01-24
2	2021-01-13	2021-06-29
3	2021-06-22	2022-01-08

- Memperbaiki value
Contoh: "1.X" menjadi "1".
- Imputasi Kolom berdasarkan min dan max Date dari suatu Patch pada dataset training yang tersedia.

DATASET GAMES

```
RangeIndex: 12961 entries, 0 to 12960
Data columns (total 19 columns):
#   Column              Non-Null Count  Dtype
---  -
0   No                   12961 non-null  int64
1   GameID               12961 non-null  int64
2   MatchID              12961 non-null  int64
3   Map                  12961 non-null  object
4   Team1ID              12961 non-null  int64
5   Team2ID              12961 non-null  int64
6   Team1                12961 non-null  object
7   Team2                12961 non-null  object
8   Winner               12961 non-null  int64
9   Team1_Eco            11927 non-null  float64
10  Team1_SemiEco        11927 non-null  float64
11  Team1_SemiBuy        11927 non-null  float64
12  Team1_FullBuy        11927 non-null  float64
13  Team1_TotalRounds    12961 non-null  int64
14  Team2_Eco            11927 non-null  float64
15  Team2_SemiEco        11927 non-null  float64
16  Team2_SemiBuy        11927 non-null  float64
17  Team2_FullBuy        11927 non-null  float64
18  Team2_TotalRounds    12961 non-null  int64
```

Data Cleaning Kolom Map

Menghapus baris dengan nilai Map berupa TBD atau To Be Determined

DATASET SCORES

```
RangeIndex: 6401 entries, 0 to 6400
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   No                   6401 non-null   int64
1   MatchID              6401 non-null   int64
2   Date                 6401 non-null   object
3   Patch                5990 non-null   object
4   EventID              6401 non-null   int64
5   EventName            6401 non-null   object
6   EventStage           6401 non-null   object
7   Team1ID              6401 non-null   int64
8   Team2ID              6401 non-null   int64
9   Team1                 6401 non-null   object
10  Team2                 6401 non-null   object
11  Team1_MapScore        6401 non-null   int64
12  Team2_MapScore        6401 non-null   int64
dtypes: int64(7), object(6)
```

PlayerID	Patch	
2.0	0	omen
	1	cypher
	2	jett
	3	sova
3.0	0	breach
	...	

	Agent	ACS
0	astra	184.0
1	breach	170.0
2	brimstone	185.0
3	chamber	203.0
4	cypher	184.0
5	jett	226.0
6	kayo	186.0
7	killjoy	195.0
8	omen	185.0
9	phoenix	217.0
10	raze	232.0
11	reyna	227.0
12	sage	183.0
13	skye	185.0
14	sova	194.0
15	viper	192.0
16	yoru	213.5

Imputasi Data Kolom Agent

Menggunakan modus dari **Agent** berdasarkan pemain dengan **PlayerID** tertentu pada **Patch** tertentu.

Imputasi Kolom ACS

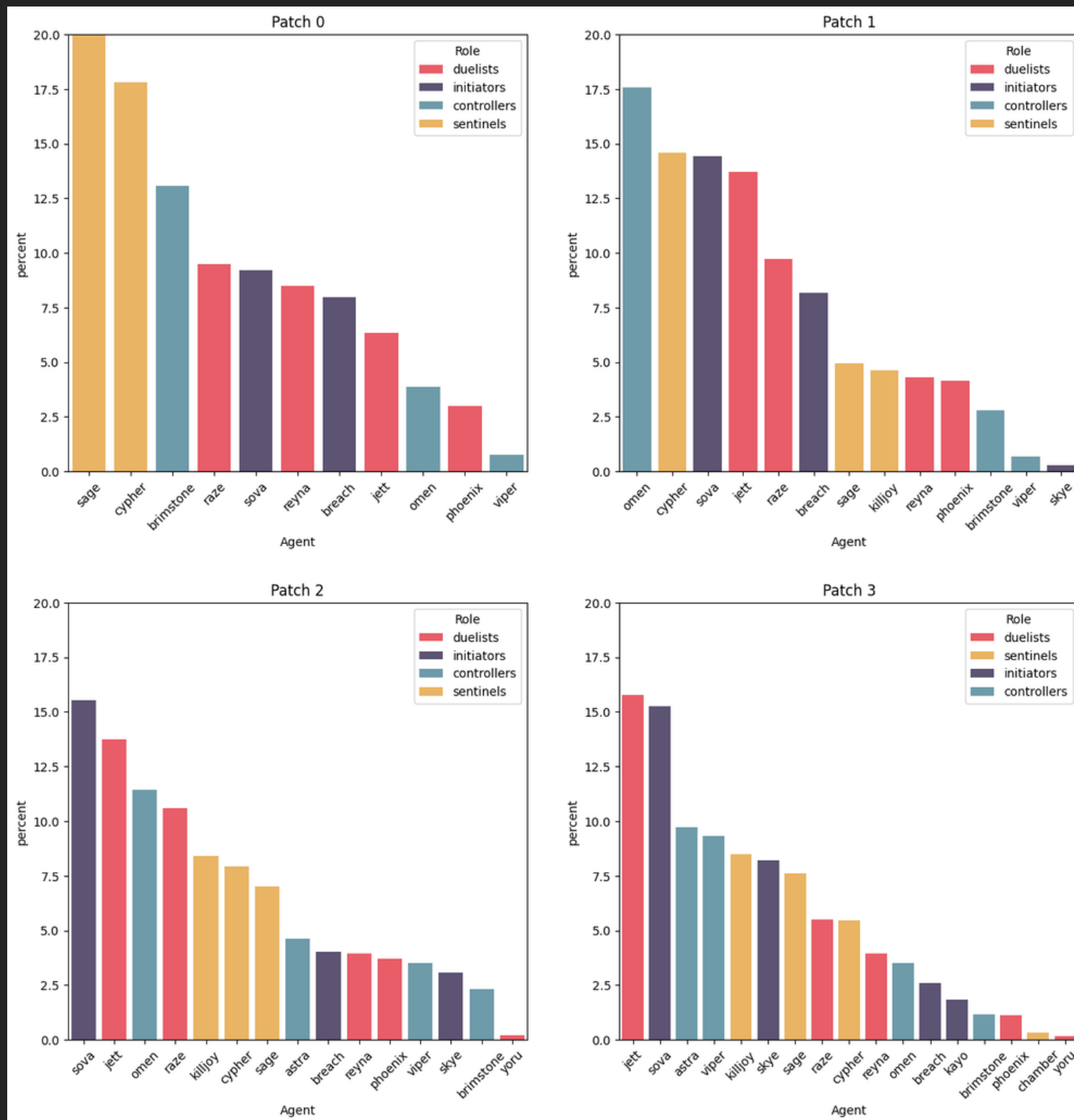
Menggunakan median dari **ACS** untuk setiap **Agent**.

EXPLORATORY DATA ANALYSIS

PROBLEM GUIDELINE

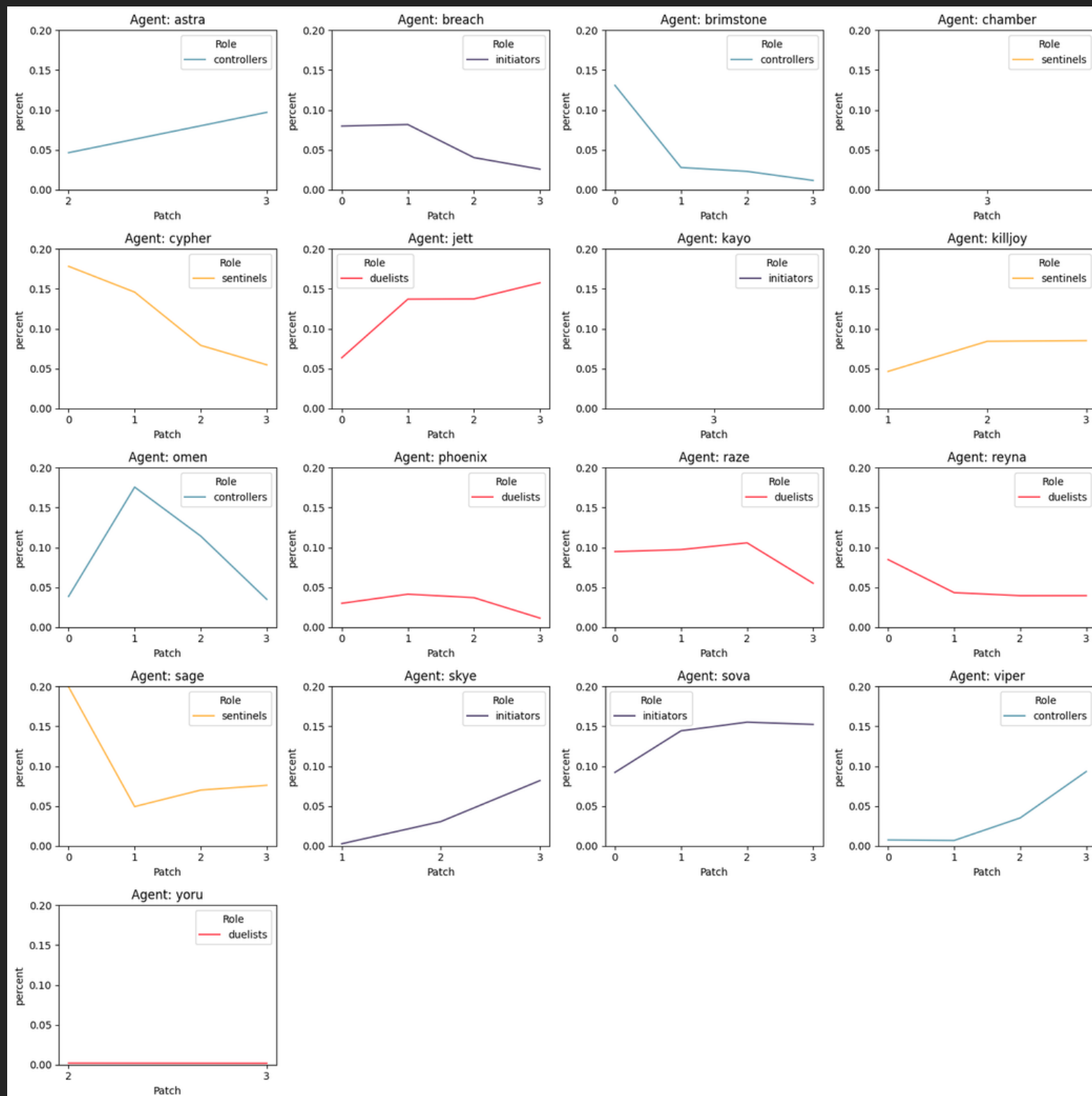
- Bagaimana distribusi Agent yang digunakan oleh pemain untuk setiap Patch 1.X, 2.X, dan 3.X? Apakah terdapat Agent yang mengalami penurunan pengguna pada patch yang berbeda?
- Bagaimana persebaran nilai ACS dari setiap Agent yang digunakan? Apakah pemilihan map yang berbeda menghasilkan persebaran nilai ACS yang berbeda juga?
- Siapa tim yang paling banyak memenangkan pertandingan grand final dari sebuah Event?
- Komposisi tim seperti apakah yang paling banyak memenangkan pertandingan?
- Bagaimana distribusi Agent yang digunakan oleh pemain untuk setiap map?
- Bagaimana statistik dari setiap Agent yang digunakan oleh pemain pada seluruh pertandingan turnamen?

DISTRIBUSI AGENT SETIAP PATCH



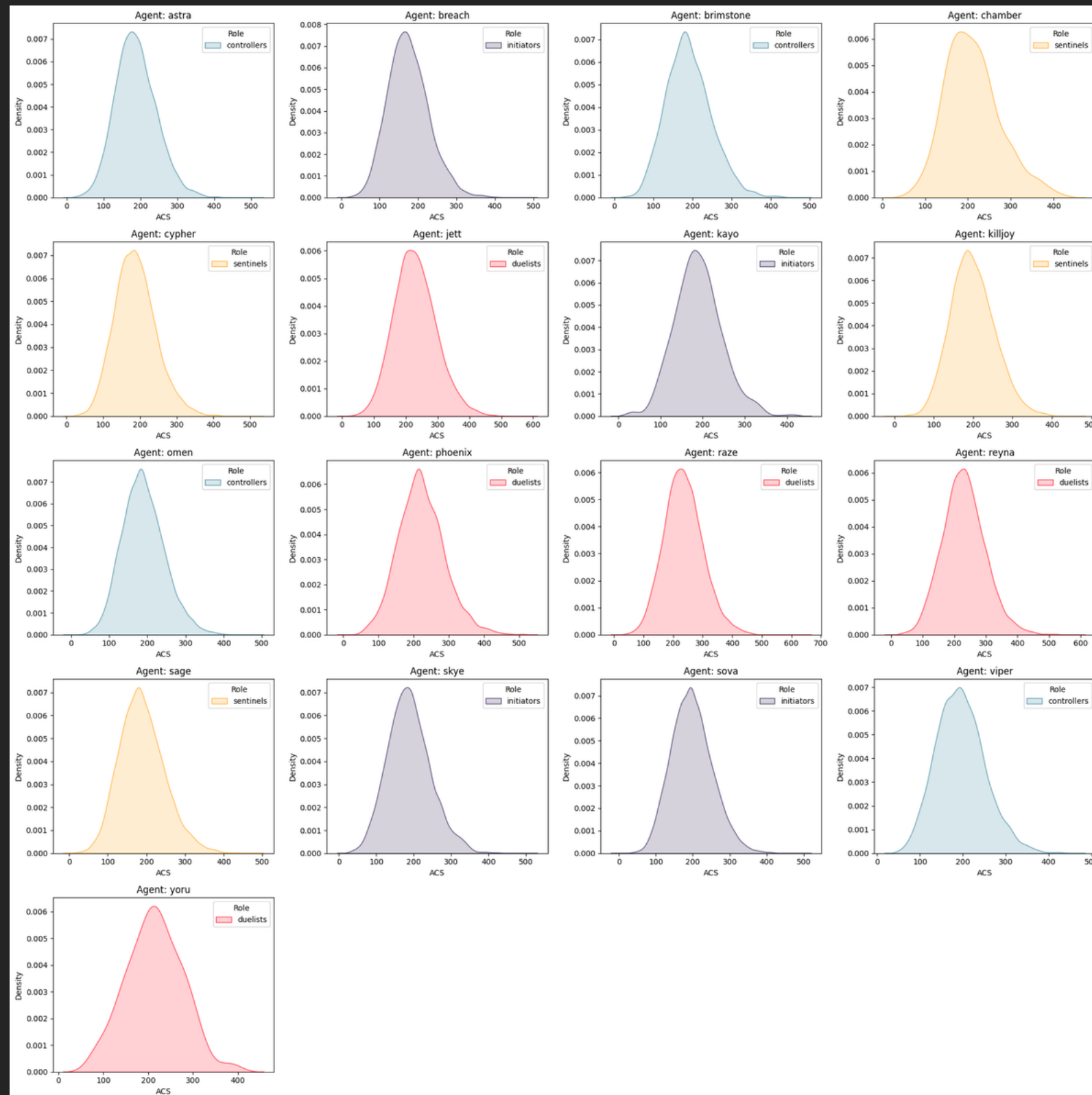
- Distribusi agent untuk setiap patch bervariasi
- Distribusi agent menandakan terdapat META (*Most Effective Tactic Available*) pada setiap patch berkaitan dengan pemilihan agent
- Agent-agent dengan persentase tertinggi untuk setiap `Patch` adalah sebagai berikut.
 - 1. Patch 0: sage
 - 2. Patch 1: omen
 - 3. Patch 2: sova
 - 4. Patch 3: jett

TREND AGENT BERDASARKAN PATCH



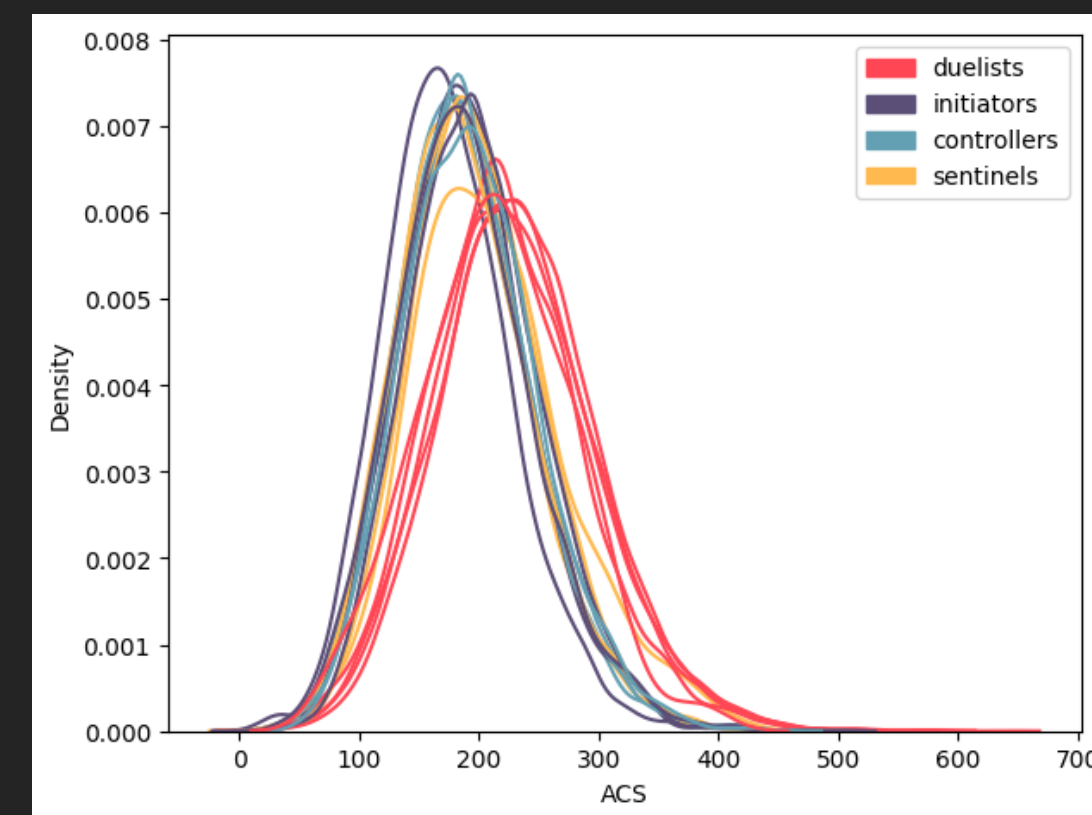
- Ada **Agent** yang mengalami penurunan pengguna pada **Patch** yang berbeda. Hal ini dapat mengindikasikan ada Agent lain yang lebih menguntungkan
- Ada **Agent** yang mengalami kenaikan pengguna pada **Patch** yang berbeda. Hal ini mengindikasikan agent tersebut memiliki keuntungan pada Patch tertentu.
- Ada **Agent** yang baru tersedia pada **Patch** tertentu saja karena adanya pembaruan (update) dari game Valorant.
- Ada berbagai faktor yang mempengaruhi trend pemilihan **Agent**, seperti Perubahan kemampuan dari **Agent** pada **Patch** tertentu dan banyak **Agent** baru yang bermunculan.

PERSEBARAN NILAI ACS AGENT



Berikut persebaran nilai **ACS** untuk setiap **Agent**:

- Persebaran **ACS** yang diperoleh cenderung normal
- Mayoritas nilai berada pada angka 200
- Terlihat bahwa terdapat beberapa **Agent** yang mencapai nilai **ACS** yang tinggi maupun rendah
- **Agent** dengan **Role** yang sama cenderung memiliki nilai **ACS** yang sama
- **Agent** dengan **Role** duelist memiliki rata-rata **ACS** yang lebih tinggi dibandingkan dengan **Agent** dengan **Role** yang lain



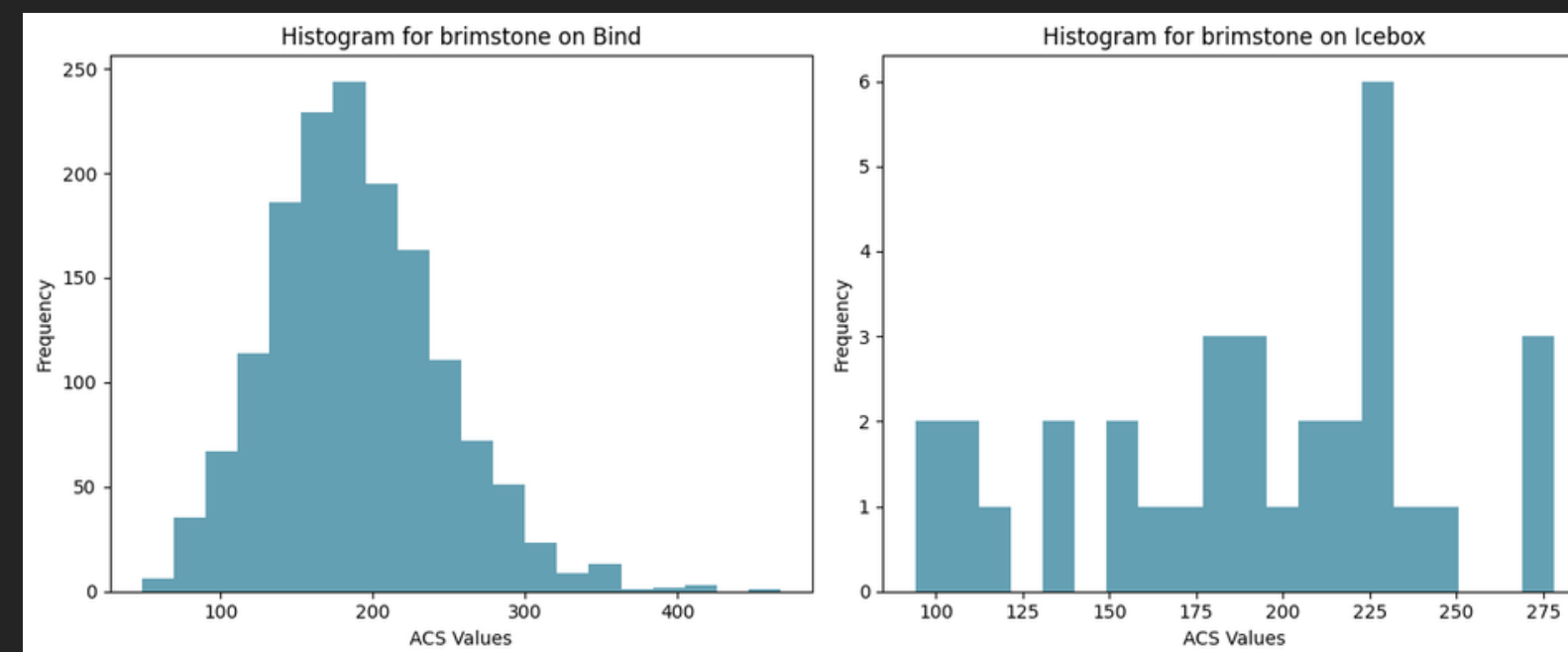
PERSEBARAN NILAI ACS AGENT



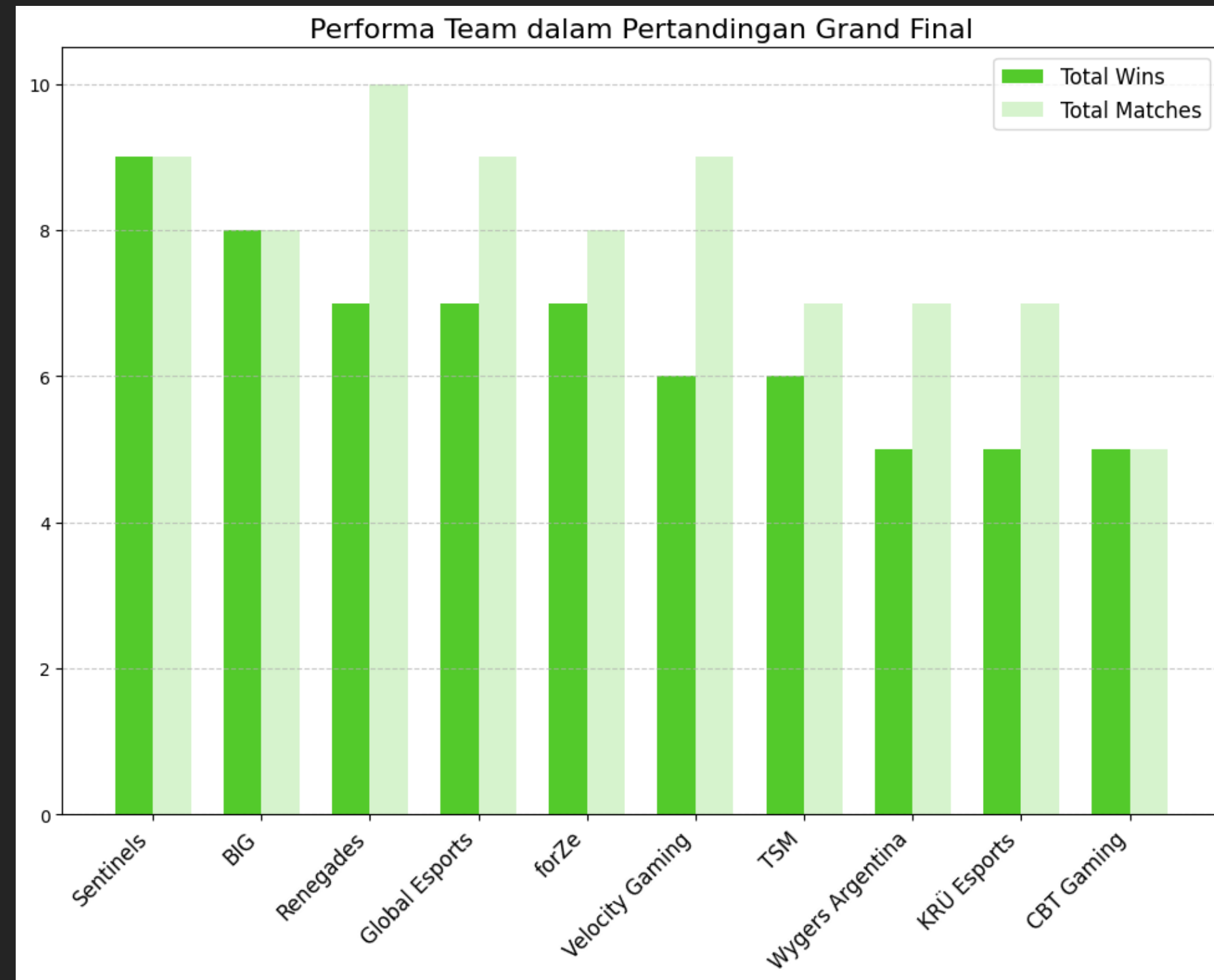
Akan tetapi pada saat kita lihat berdasarkan **Map** yang dimainkan contohnya **Agent** Brimstone:

- **Agent** mendapatkan nilai **ACS** sekitar 200 lebih sering pada **Map** Bind daripada **Map** lainnya
- **Agent** mendapatkan nilai **ACS** tertinggi pada **Map** Bind
- **Agent** mendapatkan nilai **ACS** terendah pada **Map** Split dan Haven

Dengan begitu, pemilihan **Map** dapat menyebabkan perubahan terhadap persebaran nilai **ACS**



PERFORMA TIM TERATAS PADA PERTANDINGAN GRAND FINAL



Dari hasil visualisasi tersebut, dapat disimpulkan bahwa **Tim Sentinel's** memiliki performa paling baik dalam pertandingan grand final. Hal tersebut dikarenakan **Tim Sentinel's** berhasil memenangkan semua pertandingan grand final yang mereka alami dengan total 9 kemenangan dari 9 pertandingan grand final, disusul oleh **Tim BIG** dengan total 8 kemenangan dari 8 pertandingan. Di sisi lain, **Tim Renegades** memegang jumlah pertandingan grand final tertinggi dengan 10 pertandingan namun hanya dapat memenangkan 7 pertandingan saja.

KOMPOSISI TIM BERDASARKAN AGENT

- Komposisi Agent bervariasi
- Terdapat agent yang selalu dipilih dalam kombinasi dari sebuah tim yaitu **Jett** dan **Sova** hal ini mengindikasikan bahwa kedua agent tersebut selalu masuk ke dalam META.
- Terdapat agent yang tidak pernah terpilih dalam komposisi sebuah tim, yaitu **Kayo, Chamber, Brimstone, dan Yoru.**

10 Komposisi Tim yang Paling Sering Memenangkan Pertandingan berdasarkan Agent

jett	sova	astra	kayo	killjoy	skye	cypher	viper	sage	reyna	chamber	raze	breach	omen	brimstone	phoenix	yoru	Total Usages
Used	Used	Unused	Unused	Unused	Unused	Used	Unused	Unused	Unused	Unused	Used	Unused	Used	Unused	Unused	Unused	451
Used	Used	Unused	Unused	Used	Unused	Unused	Used	Used	Unused	Unused	Unused	Unused	Unused	Unused	Unused	Unused	428
Used	Used	Unused	Unused	Unused	Unused	Used	Unused	Unused	Unused	Unused	Unused	Unused	Used	Unused	Used	Unused	401
Used	Used	Unused	Unused	Unused	Unused	Unused	Used	Used	Used	Unused	Unused	Unused	Unused	Unused	Unused	Unused	388
Used	Used	Unused	Unused	Unused	Unused	Used	Unused	Unused	Unused	Unused	Unused	Used	Used	Unused	Unused	Unused	380
Used	Used	Used	Unused	Used	Used	Unused	Unused	Unused	Unused	Unused	Unused	Unused	Unused	Unused	Unused	Unused	360
Used	Used	Unused	Unused	Unused	Unused	Used	Unused	Unused	Used	Unused	Unused	Unused	Used	Unused	Unused	Unused	348
Used	Used	Unused	Unused	Used	Unused	Unused	Unused	Unused	Used	Unused	Unused	Unused	Used	Unused	Unused	Unused	238
Used	Used	Unused	Unused	Unused	Unused	Used	Unused	Used	Unused	Unused	Unused	Unused	Used	Unused	Unused	Unused	236
Used	Used	Unused	Unused	Used	Unused	Unused	Unused	Used	Unused	Unused	Unused	Unused	Used	Unused	Unused	Unused	225

Color Legend: ■ Used ■ Unused

KOMPOSISI TIM BERDASARKAN ROLE

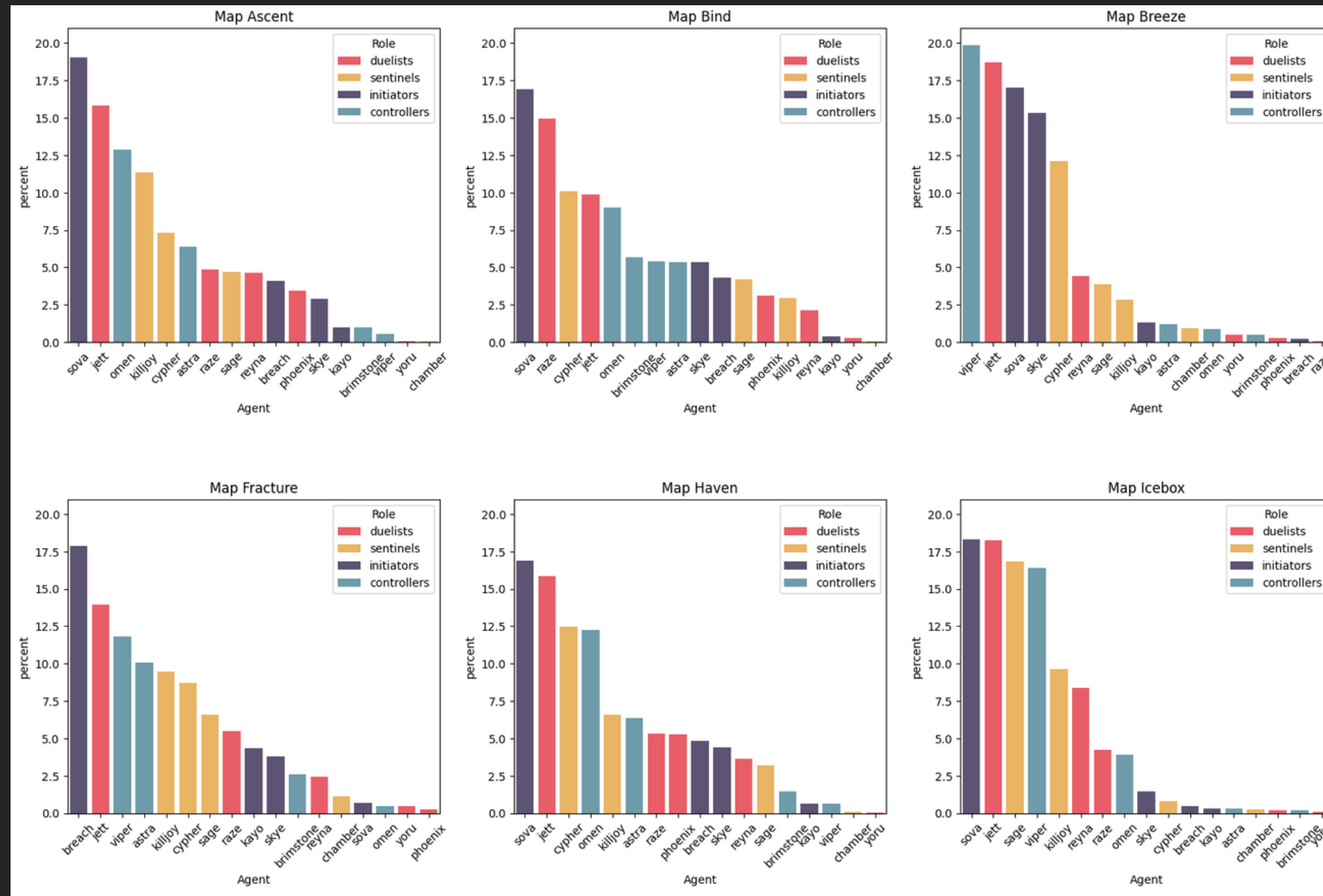
10 Komposisi Tim yang Paling Sering Memenangkan Pertandingan berdasarkan Role

duelists	initiators	controllers	sentinels	Total Usages
■	■	■	■	4687
■	■	■	■	2746
■	■	■	■	2364
■	■	■	■	633
■	■	■	■	613
■	■	■	■	420
■	■	■	■	311
■	■	■	■	214
■	■	■	■	109
■	■	■	■	108

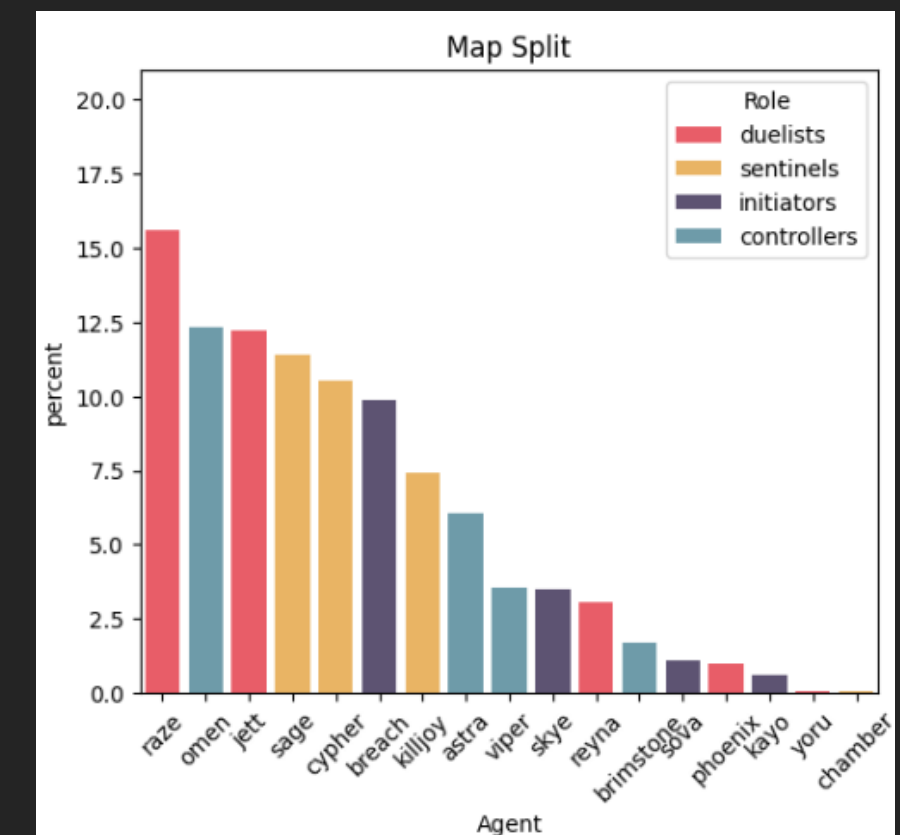
Color Legend: ■ Unused ■ Used 1 Times
 ■ Used 2 Times ■ Used 3 Times

- Terdapat kombinasi role yang paling banyak digunakan dengan total penggunaan sebanyak 4687 yaitu **2 Duelists, 1 Initiators, 1 Controllers, dan 1 Sentinels**. Hal tersebut sesuai dengan kompoisis tim berdasarkan agent pada visualisasi sebelumnya
- Tidak semua role terpilih dalam kombinasi sebuah tim, tetapi terdapat satu role yang selalu dipilih yaitu **Controllers**

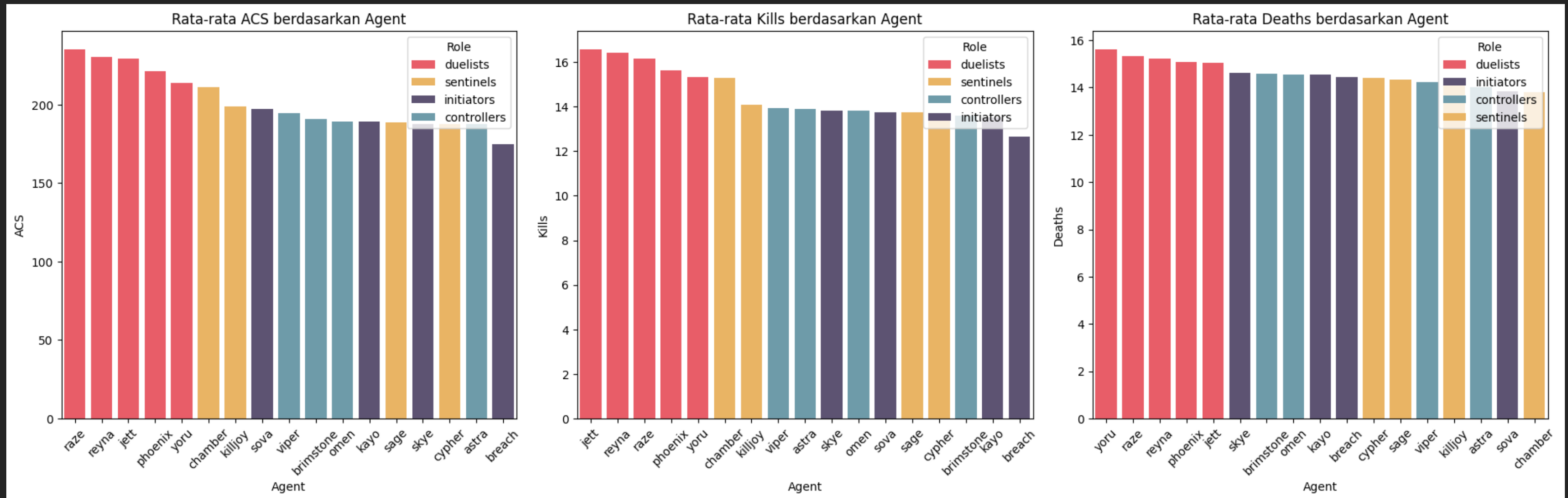
PERSENTASE AGENT BERDASARKAN MAP



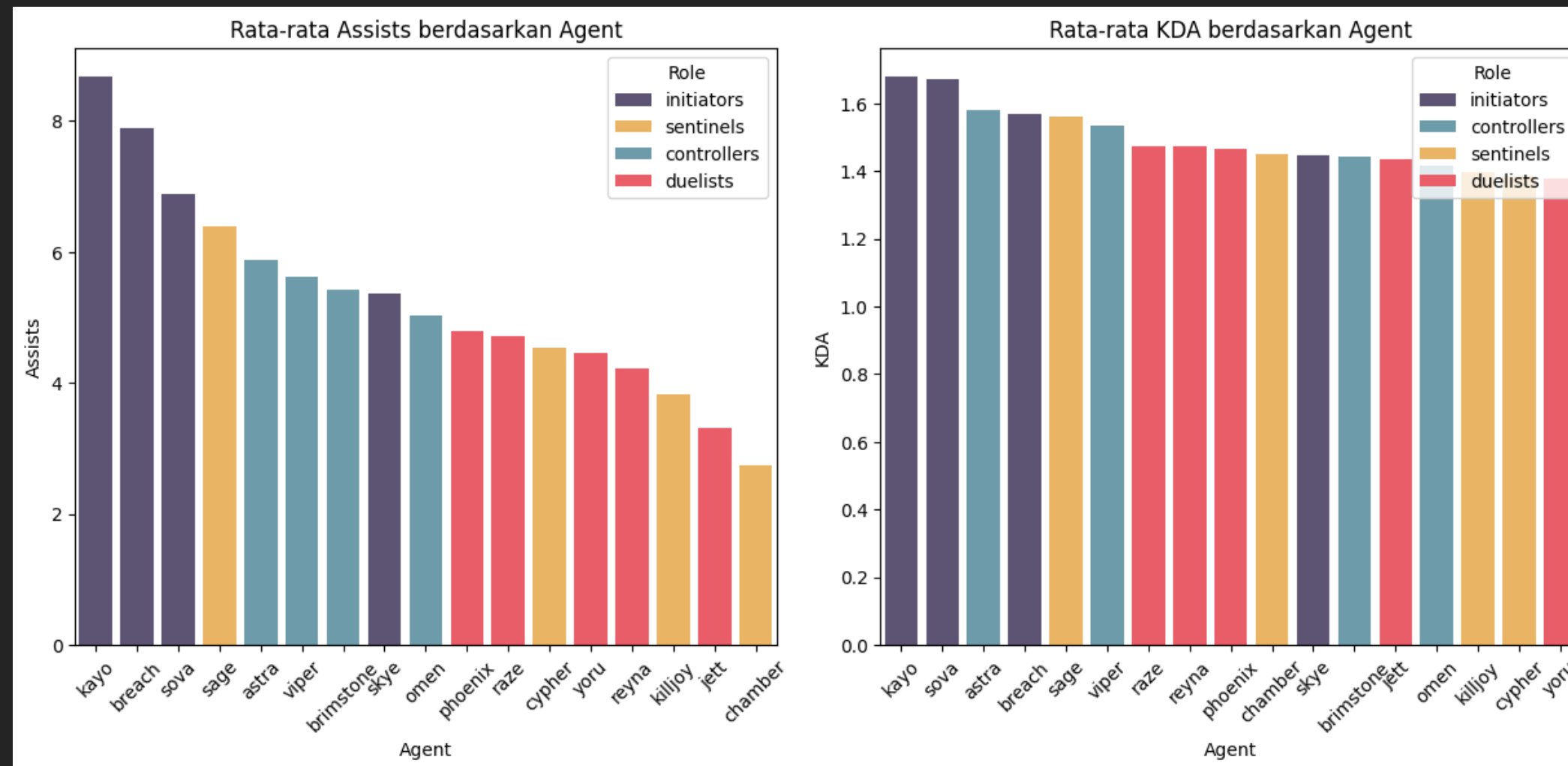
- **Agent** yang digunakan oleh pemain untuk setiap **Map** cukup bervariasi.
- Distribusi menandakan adanya keuntungan yang dimiliki oleh **Agent** tertentu pada suatu **Map** untuk meningkatkan kemungkinan tim untuk memenangkan pertandingan.
- Agent-agent dengan persentase tertinggi untuk setiap **Map** adalah sebagai berikut.
 1. Ascent: sova
 2. Bind: sova
 3. Breeze: viper
 4. Fracture: breach
 5. Haven: sova
 6. Icebox: sova
 7. Split: raze



RATA-RATA ACS, KILLS, DEATHS, ASSISTS, DAN KDA SETIAP AGENT



RATA-RATA ACS, KILLS, DEATHS, ASSISTS, DAN KDA SETIAP AGENT

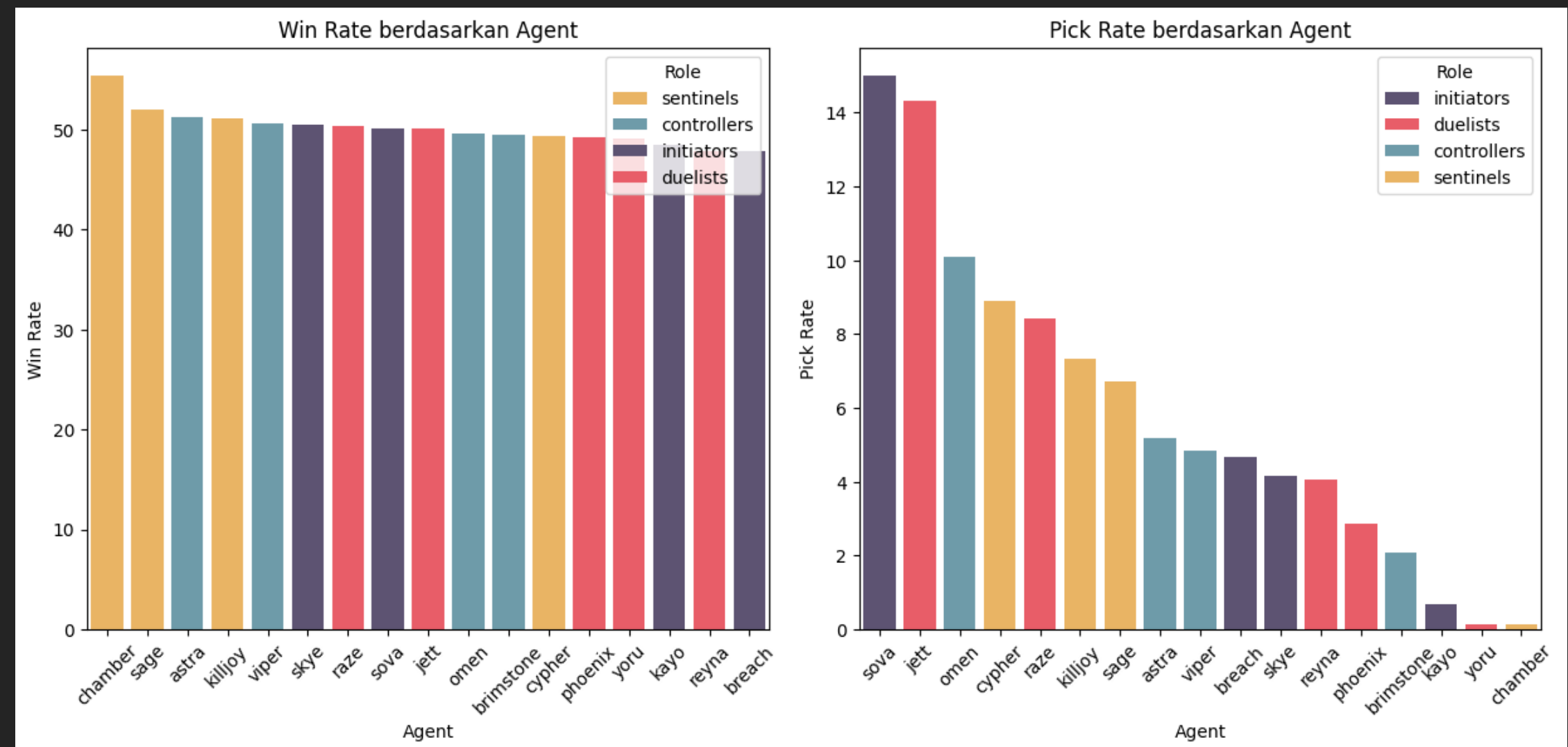


- Berdasarkan rata-rata ACS, Kills, dan Deaths, agent yang memiliki nilai rata-rata tertinggi didominasi oleh agent dengan role **duelists**. Hal tersebut sesuai dengan role **duelists** itu sendiri yang berperan untuk membuka jalan bagi rekan satu tim untuk masuk ke wilayah musuh dan berfokus untuk mendapatkan kill dan memberikan damage kepada musuh
- Berdasarkan rata-rata Assists, agent yang memiliki nilai rata-rata tertinggi didominasi oleh agent dengan role **initiators**. Hal tersebut sesuai dengan peran **initiators** yaitu untuk menggali informasi mengenai musuh dan menggunakan *ability* mereka untuk membantu rekan satu tim ketika berhadapan dengan musuh

Sumber: <https://www.zleague.gg/theportal/all-valorant-roles-a-valorant-class-guide/>

WIN RATE DAN PICK RATE SETIAP AGENT

- Agent dengan pick rate tertinggi merupakan agent yang termasuk dalam komposisi tim berdasarkan agent yang sering digunakan yaitu **Jett, Sova, Omen, Cypher,** dan **Raze**.
- **Chamber** merupakan agent dengan win rate tertinggi, namun hal tersebut disebabkan **Chamber** termasuk agent yang baru dikenalkan oleh Valorant dan memiliki pick rate terendah.



DATA PREPROCESSING

DATASET MATCHES

```
df_matches.isna().sum()
```

No	0
MatchID	0
Date	0
Patch	0
EventID	0
EventName	0
EventStage	0
Team1ID	0
Team2ID	0
Team1	0
Team2	0
Team1_MapScore	0
Team2_MapScore	0
dtype:	int64

Transformasi Kolom Date

Dengan selisih hari antara **Date** dari pertandingan dengan **Date** dari perilisan game Valorant ke publik.

Buat Kolom Team_MapScore

Dengan menghitung rata-rata antara **Team1_MapScore** dan **Team2_MapScore** untuk mengurangi dimensi fitur.

DATASET GAMES

```
df_games.isna().sum()
```

No	0
GameID	0
MatchID	0
Map	0
Team1ID	0
Team2ID	0
Team1	0
Team2	0
Winner	0
Team1_Eco	974
Team1_SemiEco	974
Team1_SemiBuy	974
Team1_FullBuy	974
Team1_TotalRounds	0
Team2_Eco	974
Team2_SemiEco	974
Team2_SemiBuy	974
Team2_FullBuy	974
Team2_TotalRounds	0
dtype:	int64

Imputasi Kolom Team1_Eco, Team1_SemiEco, Team1_SemiBuy, Team1_FullBuy

Menggunakan median dari masing-masing kolom per **Team** dan median secara global.

Buat Kolom Team_Eco, Team_SemiEco, Team_SemiBuy, dan Team_FullBuy

Dengan menghitung rata-rata antara **Team1_X** dan **Team2_X** untuk mengurangi dimensi fitur dengan **X = Eco, SemiEco, SemiBuy, dan FullBuy**.

DATASET SCORES

```
df_scores.isna().sum()

No      0
GameID  0
PlayerID 0
PlayerName 0
TeamAbbreviation 1882
Agent    607
ACS       99
Kills    490
Deaths   490
Assists  490
PlusMinus 490
KAST_Percent 124876
ADR      7612
HS_Percent 8206
FirstKills 530
FirstDeaths 8202
FKFD_PlusMinus 8202
Num_2Ks 8932
Num_3Ks 8932
Num_4Ks 8932
Num_5Ks 8932
OnevOne 8932
OnevTwo 8932
OnevThree 8932
OnevFour 8932
OnevFive 8932
Econ     8932
Plants   8932
Defuses  8932
dtype: int64
```

Imputasi kolom TeamAbbreviation

Dilakukan dengan memperhatikan **TeamAbbreviation** berdasarkan kolom **GameID** dan **Agent** yang digunakan oleh pemain.

Drop Kolom

Kolom-Kolom yang mengandung null dan tidak dapat diimputasi di-drop. Selain itu, kolom **KAST_PERCENT** diabaikan karena terlalu banyak nilai null.

Buat Kolom Num_Ks

Menghitung rata-rata berbobot antara **Num_2Ks**, **Num_3Ks**, **Num_4Ks**, dan **Num_5Ks** untuk mengurangi dimensi dari fitur.

Buat Kolom OnevX

Menghitung rata-rata berbobot antara **OnevOne**, **OnevTwo**, **OnevThree**, **OnevFour**, dan **OnevFive** untuk mengurangi dimensi dari fitur.

DATASET MERGE

Data columns (total 66 columns):

#	Column	Non-Null Count	Dtype
0	No_x	117679 non-null	int64
1	MatchID	117679 non-null	int64
2	Date	117679 non-null	int64
3	Patch	117679 non-null	object
4	EventID	117679 non-null	int64
5	EventName	117679 non-null	object
6	EventStage	117679 non-null	object
7	Team1ID	117679 non-null	int64
8	Team2ID	117679 non-null	int64
9	Team1_x	117679 non-null	object
10	Team2_x	117679 non-null	object
11	Team1_MapScore	117679 non-null	int64
12	Team2_MapScore	117679 non-null	int64
13	Team_MapScore	117679 non-null	float64
14	No_y	117679 non-null	int64
15	GameID	117679 non-null	int64
16	Map	117679 non-null	object
17	Team1_y	117679 non-null	object
18	Team2_y	117679 non-null	object
19	Winner	117679 non-null	int64
...			
64	OnevX	117679 non-null	float64
65	Total_Agent	117679 non-null	int64

Buat Kolom Total_Agent

Menghitung **Total_Agent** yang tersedia pada suatu **Patch** tertentu.

CLUSTER ANALYSIS

ANALISIS PERFORMA DAN PENGUNAAN AGENT SETIAP MAP

Tujuan

Dalam dataset ini kita mendapatkan data seputar "Valorant Pro Matches", yang mana menampilkan hasil dari setiap pertandingan dari suatu event atau acara tertentu. Dengan data tersebut kita dapat memanfaatkannya untuk membentuk cluster. Analisis cluster yang kami pilih adalah kami ingin mengetahui performa dan penggunaan agent-agent di setiap map.

Target

Dengan cluster tersebut, target kami adalah orang-orang META analyst atau Valorant Esport Analyst dan penasehat strategi pada tim yang ingin mengetahui seberapa baik dan seberapa sering terpilih suatu agent di map tersebut.

PREPROCESSING

```
df_cluster = df_matches_games_scores[['ACS',  
                                       'Kills',  
                                       'Deaths',  
                                       'Assists',  
                                       'FirstKills',  
                                       'FirstDeaths',  
                                       'Econ',  
                                       'Plants',  
                                       'Defuses',  
                                       'Map',  
                                       'Agent']]
```

Memilih Kolom untuk Cluster

Kita awali dengan menentukan kolom-kolom apa yang akan kita gunakan untuk memberikan identitas terhadap data yang akan kita cluster

PREPROCESSING

```
numerical = ['ACS', 'Kills', 'Deaths', 'Assists',  
            'FirstKills', 'FirstDeaths', 'Econ', 'Plants', 'Defuses']  
  
Q1 = df_cluster[numerical].quantile(0.25)  
Q3 = df_cluster[numerical].quantile(0.75)  
  
IQR = Q3 - Q1  
lower_limit = Q1 - 1.5*IQR  
upper_limit = Q3 + 1.5*IQR  
  
outliers = (df_cluster[numerical] < lower_limit) | (df_cluster[numerical] > upper_limit)  
  
percentage_outliers = outliers.mean() * 100  
  
print("Percentage of outliers for each attribute:")  
print(percentage_outliers)
```

```
Percentage of outliers for each attribute:  
ACS          1.267856  
Kills        1.109799  
Deaths       1.591618  
Assists      1.462453  
FirstKills   2.066639  
FirstDeaths  1.299297  
Econ         2.290128  
Plants       3.150945  
Defuses     1.165034  
dtype: float64
```

```
df_cluster = df_cluster[~outliers.any(axis=1)]
```

Pengecekan Outliers

Kita perlu mengecek kembali untuk memastikan bahwa data yang kita pilih tidak memiliki jumlah outlier yang besar terutama saat kita menggunakan K-Means Clustering

Drop Outliers

Dikarenakan jumlah outlier sedikit hanya hingga 1% sampai 3% maka kita bisa memilih untuk drop outliers tersebut

PREPROCESSING

```
from sklearn.preprocessing import StandardScaler, LabelEncoder

df_cluster_copy = df_cluster.copy()

scaler = StandardScaler()
df_cluster_copy[numerical] = scaler.fit_transform(df_cluster_copy[numerical])

label = LabelEncoder()

df_cluster_copy['Agent'] = label.fit_transform(df_cluster['Agent'])
df_cluster_copy
```

Standarisasi

Kita perlu juga melakukan standarisasi terhadap data-data numerik yang akan kita gunakan agar memiliki range nilai yang sama

Labeling

Untuk data kategorikal, kita akan terapkan labeling sebagai identitas baru dari data tersebut yang mana akan memudahkan penggunaan data

PREPROCESSING

Pencarian n Cluster dengan Elbow Method

Sebelum kita memproses cluster, kita dapat mencoba mencari n jumlah cluster yang baik dari data yang kita gunakan. Metode yang digunakan adalah Elbow Method. Dapat dilihat bahwa mayoritas terjadi perbelokkan drastis pada cluster jumlah 2 dan beberapa juga terjadi sedikit perbelokkan pada cluster jumlah 3. Kita dapat menggunakan hasil tersebut untuk cluster.

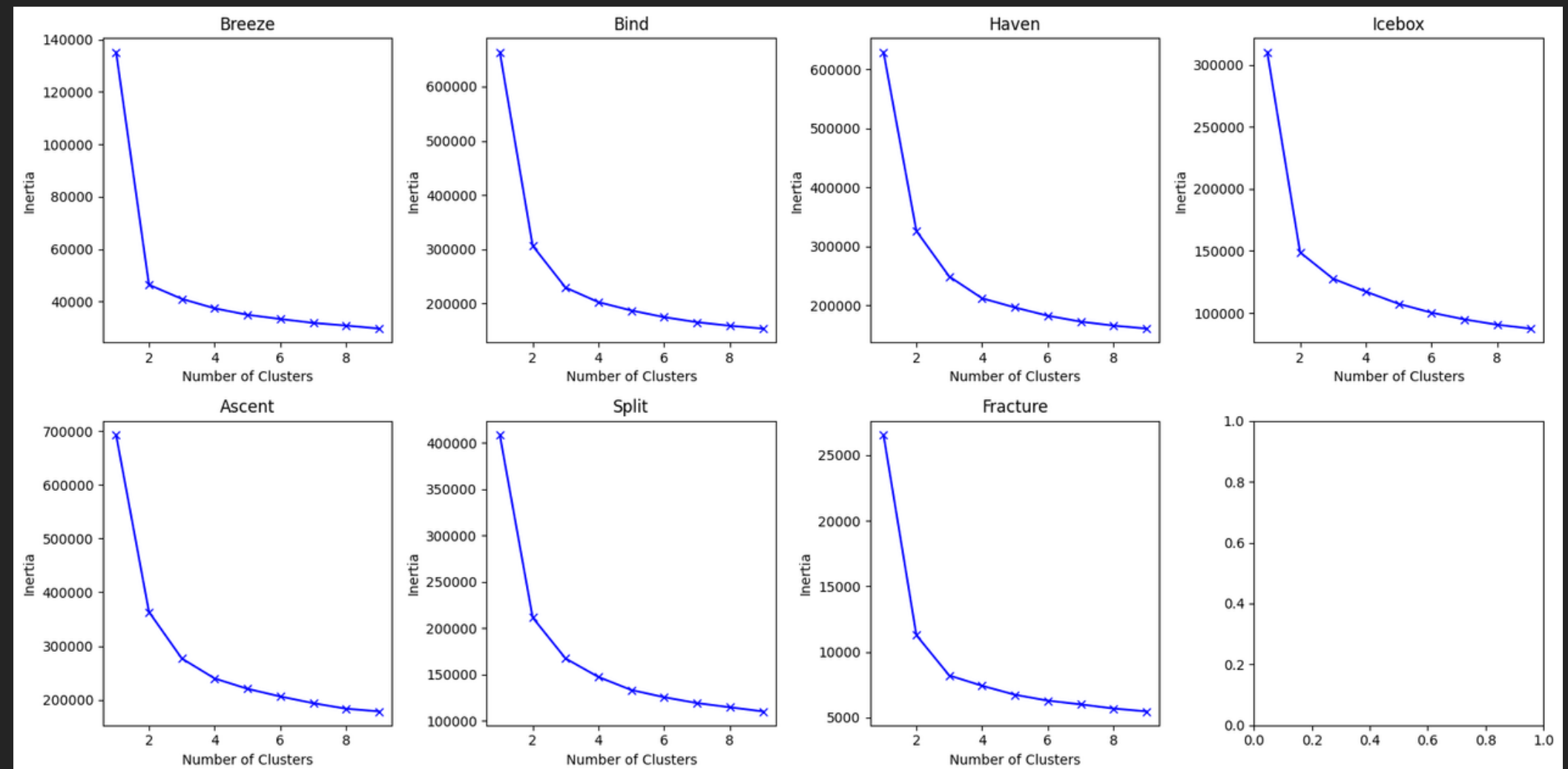
```
from sklearn.cluster import KMeans
fig, axes = plt.subplots(nrows=2, ncols=4, figsize=(16, 8))
axes = axes.flatten()

for i, map_name in enumerate(map_names):

    inertia = []
    for cluster in range(1, 10):
        km = KMeans(n_clusters=cluster)
        km = km.fit(map_dataframes[f'df_cluster_{map_name.lower()}'])
        inertia.append(km.inertia_)

    axes[i].plot(range(1, 10), inertia, 'bx-')
    axes[i].set_title(f'{map_name}')
    axes[i].set_xlabel('Number of Clusters')
    axes[i].set_ylabel('Inertia')

plt.tight_layout()
plt.show()
```



CLUSTERING #1

K-Means Cluster Map Breeze

Berdasarkan pencarian sebelumnya kita dapati bahwa untuk map Breeze terjadi perubahan yang drastis pada cluster 2 sehingga kita bisa menggunakan data map Breeze ke dalam cluster n=2

```
df_cluster_breeze = map_dataframes['df_cluster_breeze']
kmeans = KMeans(n_clusters=2, random_state=42)

cluster_assignment = kmeans.fit_predict(df_cluster_breeze)

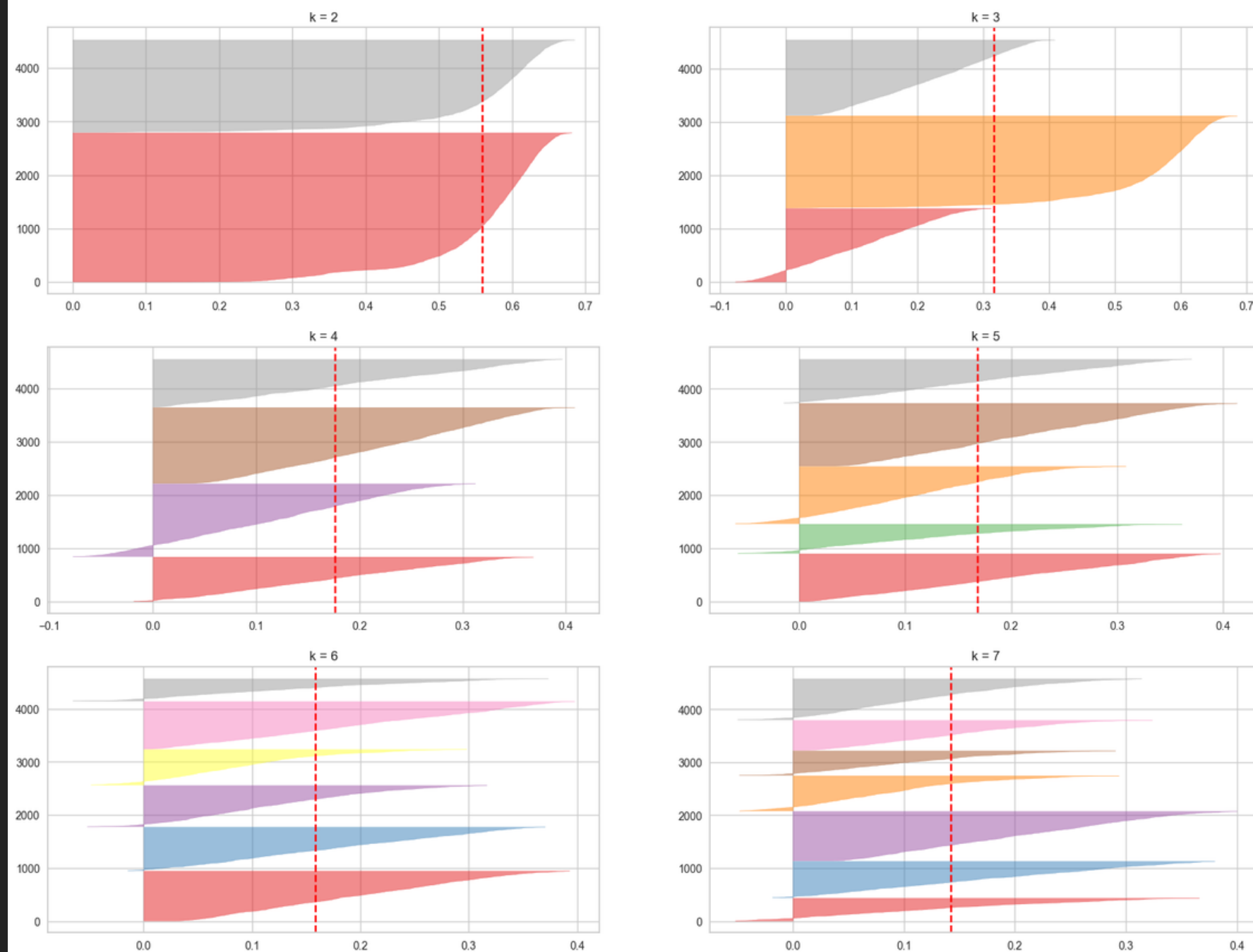
data_map_clusters = df_cluster[df_cluster['Map']=='Breeze']
data_with_clusters = data_map_clusters.copy()
data_with_clusters['Clusters'] = cluster_assignment
data_with_clusters
```

	ACS	Kills	Deaths	Assists	FirstKills	FirstDeaths	Econ	Plants	Defuses	Map	Agent	Clusters
0	313.0	24.0	10.0	3.0	4.0	4.0	74.0	0.0	0.0	Breeze	jett	0
1	227.0	16.0	10.0	7.0	1.0	1.0	67.0	2.0	0.0	Breeze	chamber	0
2	226.0	17.0	9.0	8.0	3.0	0.0	58.0	3.0	0.0	Breeze	sova	1
3	218.0	17.0	12.0	2.0	3.0	0.0	48.0	0.0	0.0	Breeze	viper	1
4	80.0	5.0	13.0	3.0	3.0	1.0	21.0	0.0	0.0	Breeze	skye	1
...	...	...	...	...	...	...	...	...	...	...	...	...
54384	145.0	11.0	16.0	5.0	1.0	0.0	42.0	0.0	0.0	Breeze	sova	1
54385	127.0	10.0	17.0	1.0	1.0	5.0	33.0	1.0	1.0	Breeze	skye	1
54386	282.0	22.0	17.0	4.0	3.0	3.0	62.0	0.0	0.0	Breeze	phoenix	0
54387	272.0	21.0	12.0	3.0	2.0	2.0	81.0	1.0	2.0	Breeze	reyna	1
54388	183.0	13.0	15.0	2.0	3.0	2.0	49.0	0.0	0.0	Breeze	jett	0

4513 rows × 12 columns

CLUSTERING #1

Visualisasi Silhouette Coefficient untuk beberapa nilai k



Evaluasi K-Means Cluster Map Breeze

Setelah kita membuat cluster, kita dapat evaluasi kembali dengan menggunakan metric evaluasi untuk clustering agar kita bisa melihat seberapa baik cluster tersebut

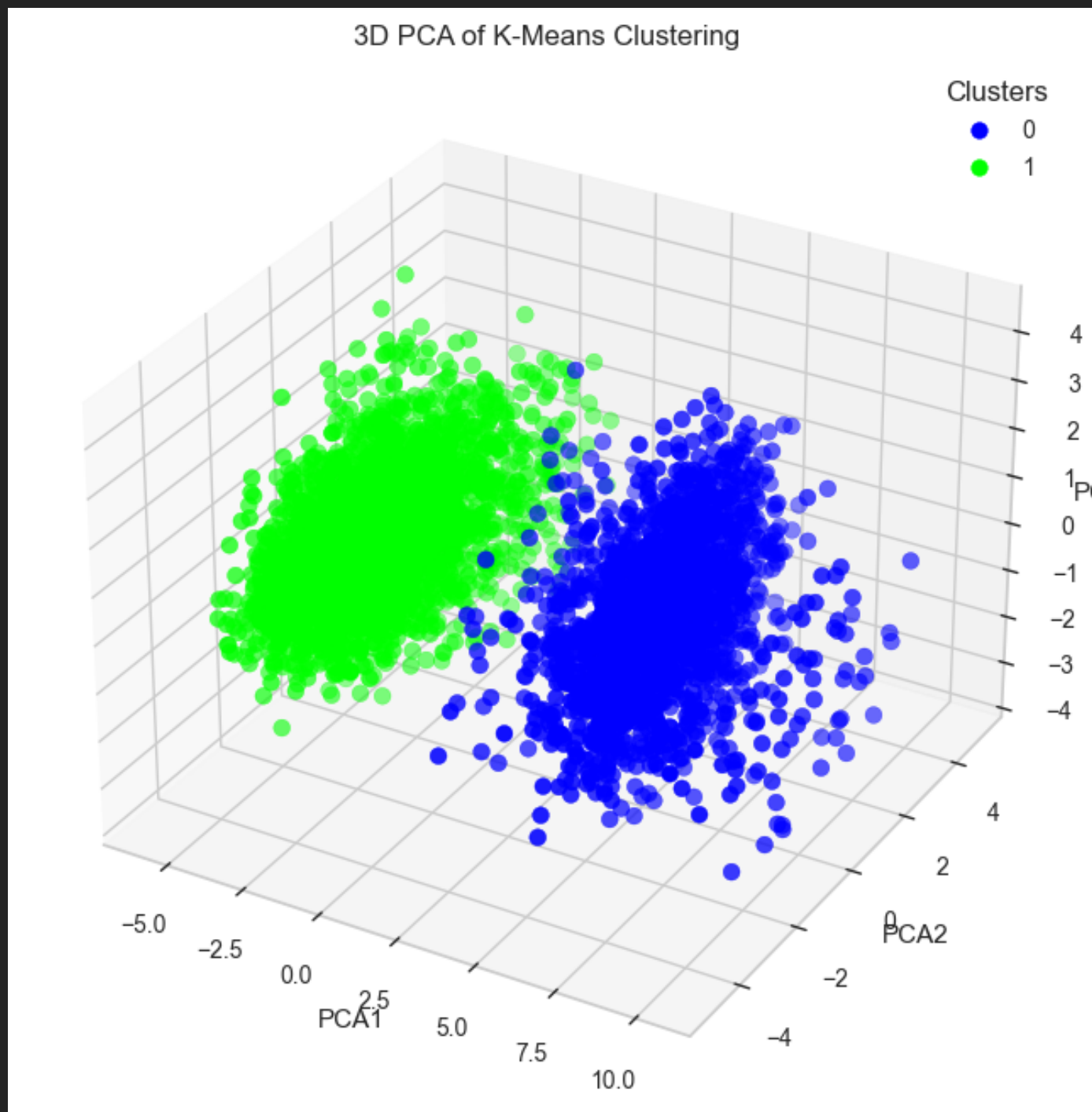
```
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score

silhouette = silhouette_score(df_cluster_breeze, cluster_assignment)
db_index = davies_bouldin_score(df_cluster_breeze, cluster_assignment)
ch_index = calinski_harabasz_score(df_cluster_breeze, cluster_assignment)

print(f"Silhouette Score: {silhouette:.2f}")
print(f"Davies-Bouldin Index: {db_index:.2f}")
print(f"Calinski-Harabasz Index: {ch_index:.2f}")
```

```
Silhouette Score: 0.56
Davies-Bouldin Index: 0.67
Calinski-Harabasz Index: 8629.50
```

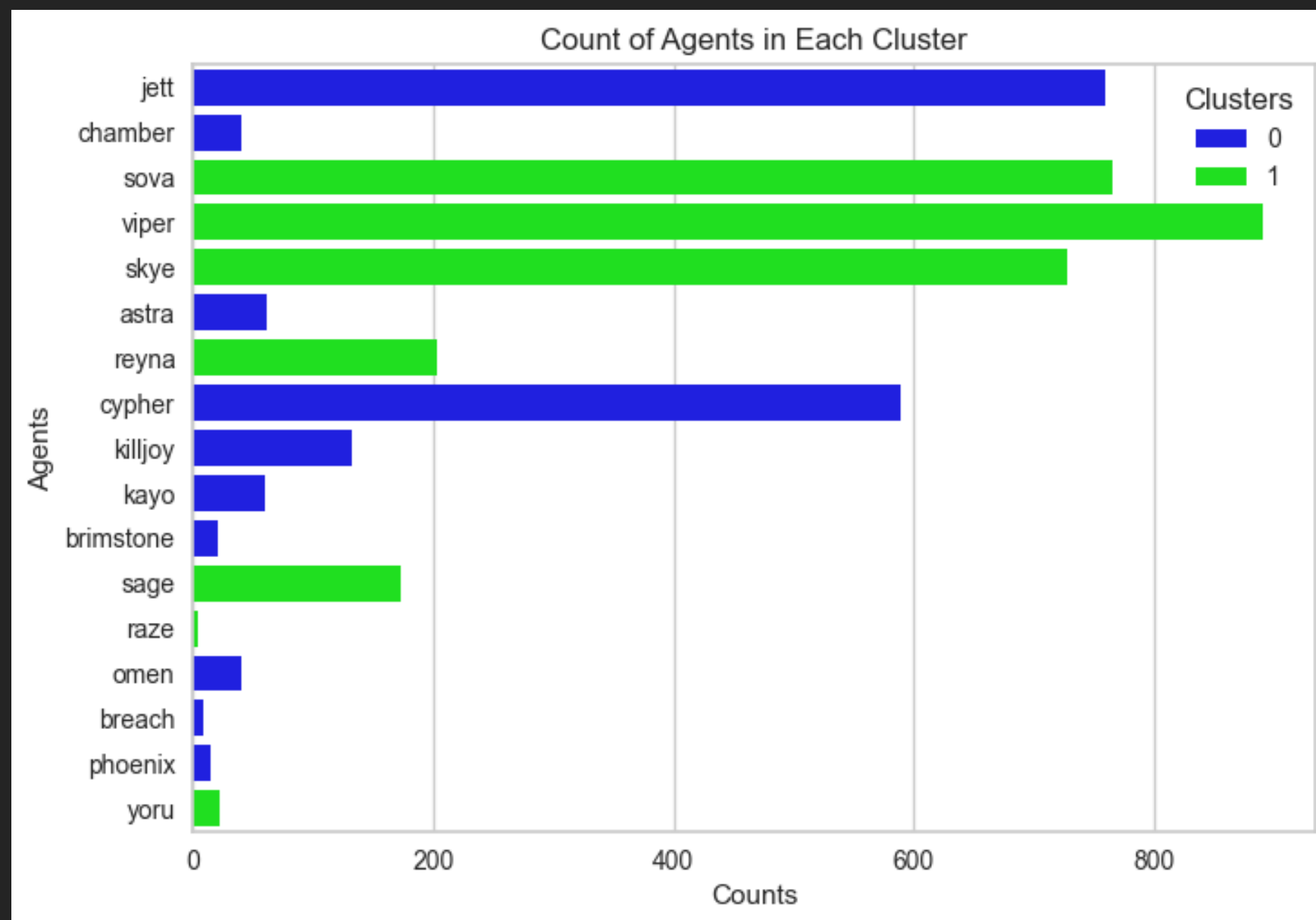
CLUSTERING #1



Visualisasi Cluster

Dengan menggunakan PCA tiga dimensi, kita dapat menggunakan data cluster dan membuat visualisasi tiga dimensi terhadap cluster seperti berikut.

CLUSTERING #1



Interpretasi Cluster: Persebaran Agent dalam Cluster

Agent paling sering dimainkan pada map adalah Viper

Agent paling jarang dimainkan pada map adalah Raze

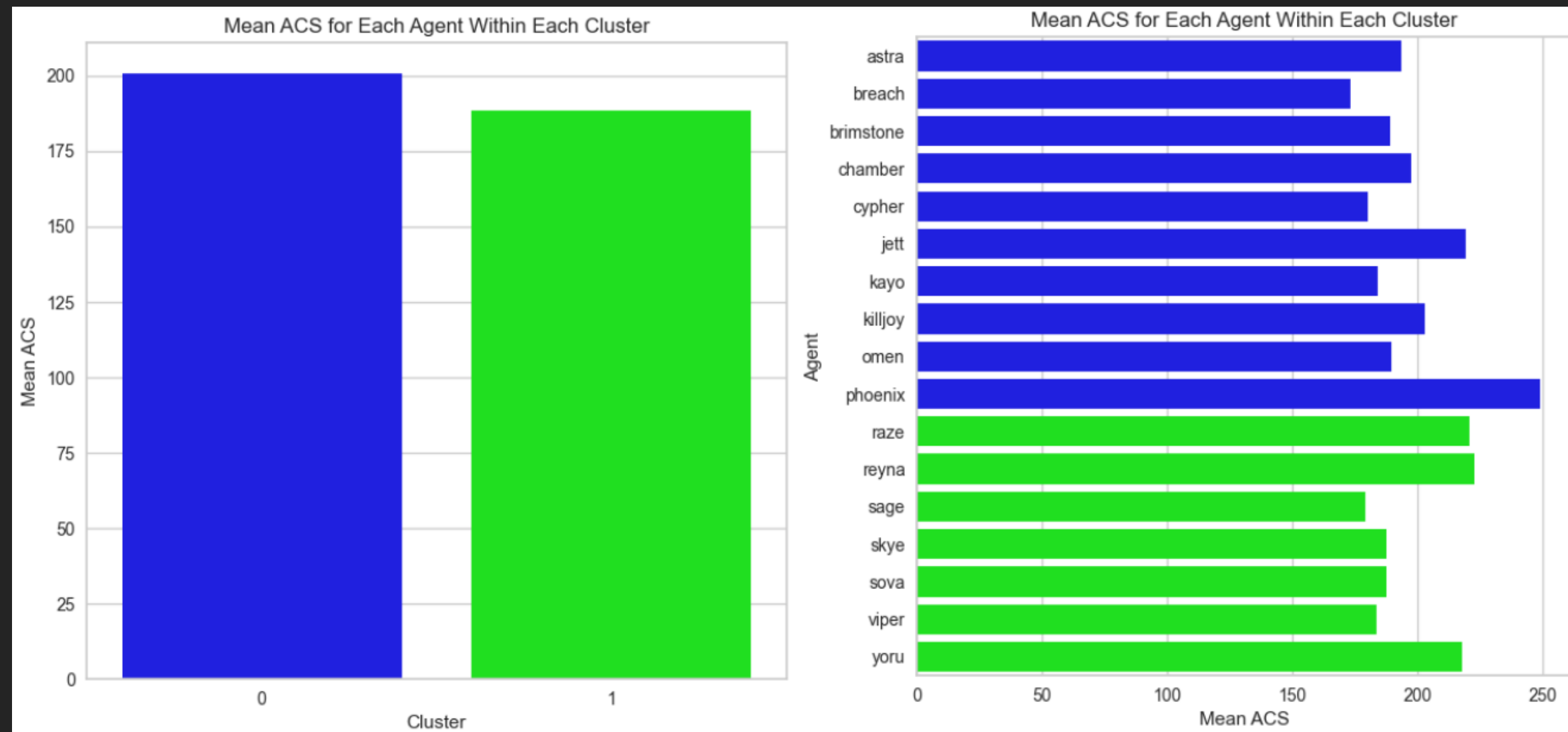
Berikut persebaran dari Agent:

Cluster 0 terdiri atas Jett, Chamber, Astra, Cypher, KillJoy, Kayo, Brimstone, Omen, Breach, Phoenix

Cluster 1 terdiri atas Sova, Viper, Skye, Reyna, Sage, Raze, Yoru

CLUSTERING #1

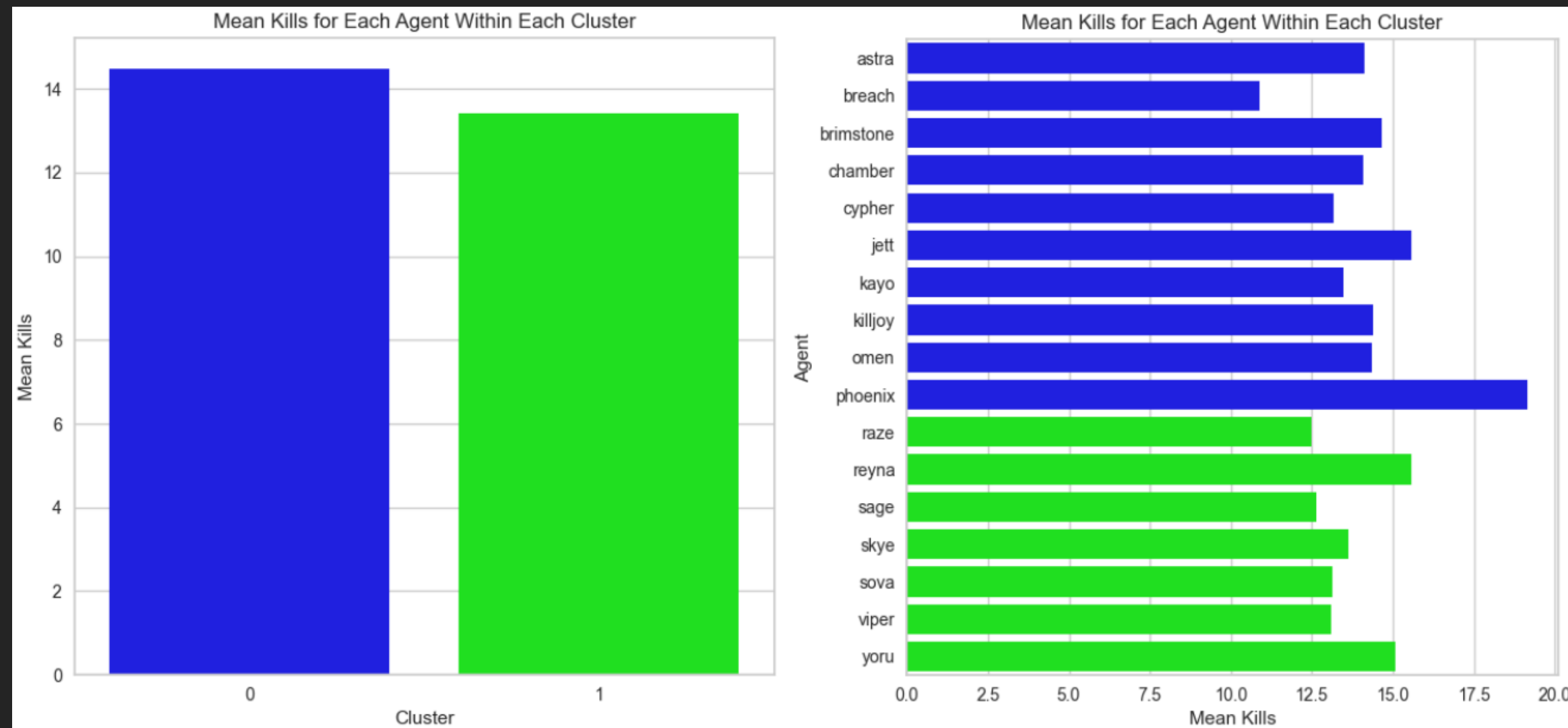
Interpretasi Cluster: Rata-rata ACS



Agent dengan rata-rata **ACS** tertinggi adalah Phoenix. Cluster 0 rata-rata **ACS** sebesar 200 dan Cluster 1 rata-rata **ACS** sebesar 185

CLUSTERING #1

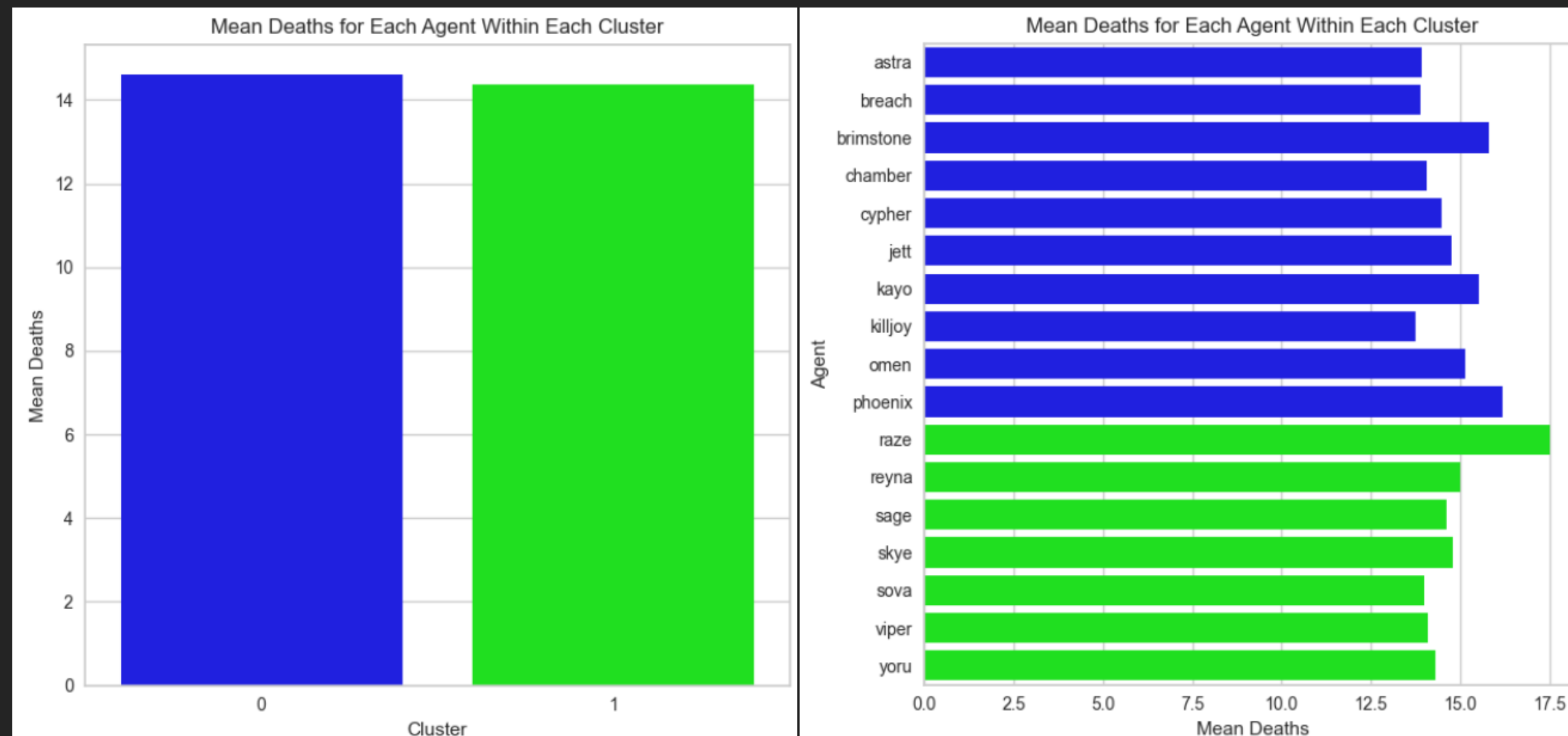
Interpretasi Cluster: Rata-rata Kills



Agent dengan rata-rata **Kills** tertinggi adalah Phoenix. Cluster 0 rata-rata **Kills** sebesar 15 dan Cluster 1 rata-rata **Kills** sebesar 13

CLUSTERING #1

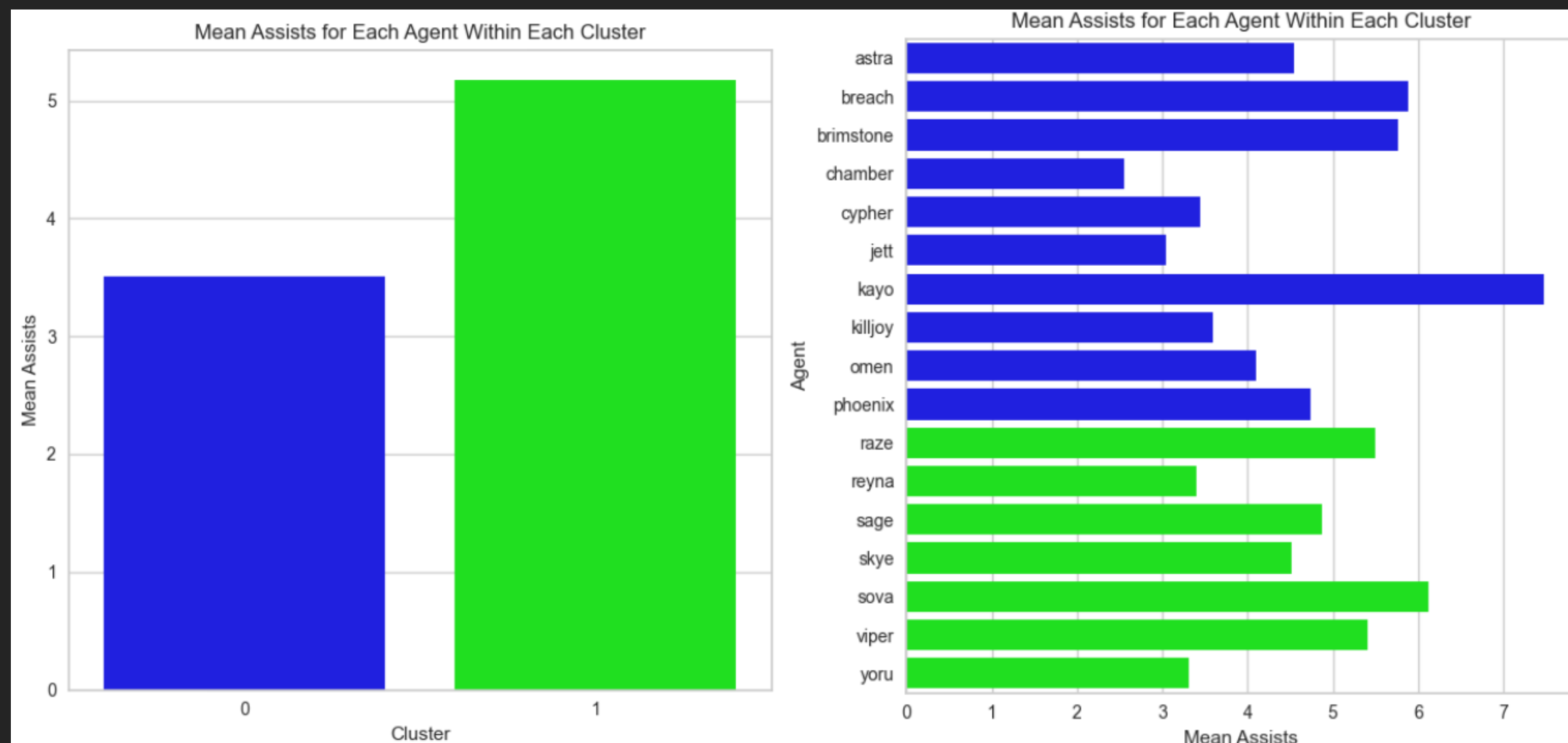
Interpretasi Cluster: Rata-rata Deaths



Agent dengan rata-rata **Deaths** tertinggi adalah Raze. Cluster 0 rata-rata **Deaths** sebesar 15 dan Cluster 1 rata-rata **Deaths** sebesar 14.5

CLUSTERING #1

Interpretasi Cluster: Rata-rata Assists



Agent dengan rata-rata **Assists** tertinggi adalah Kayo. Cluster 0 rata-rata **Assists** sebesar 3.5 dan Cluster 1 rata-rata **Assists** sebesar 5.5

CLUSTERING #1

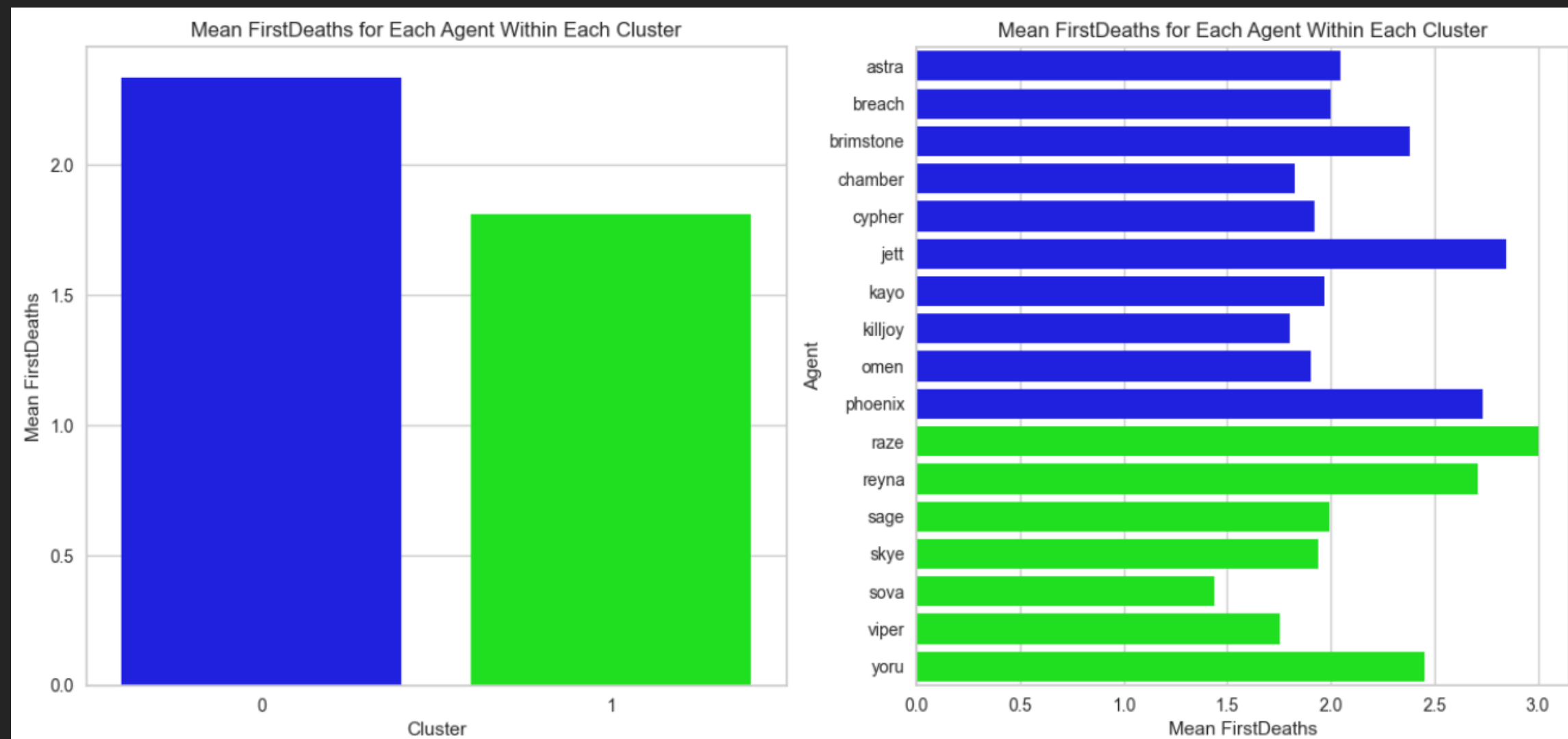
Interpretasi Cluster: Rata-rata FirstKills



Agent dengan rata-rata **FirstKills** tertinggi adalah Phoenix. Cluster 0 rata-rata **FirstKills** sebesar 2.4 dan Cluster 1 rata-rata **FirstKills** sebesar 1.55

CLUSTERING #1

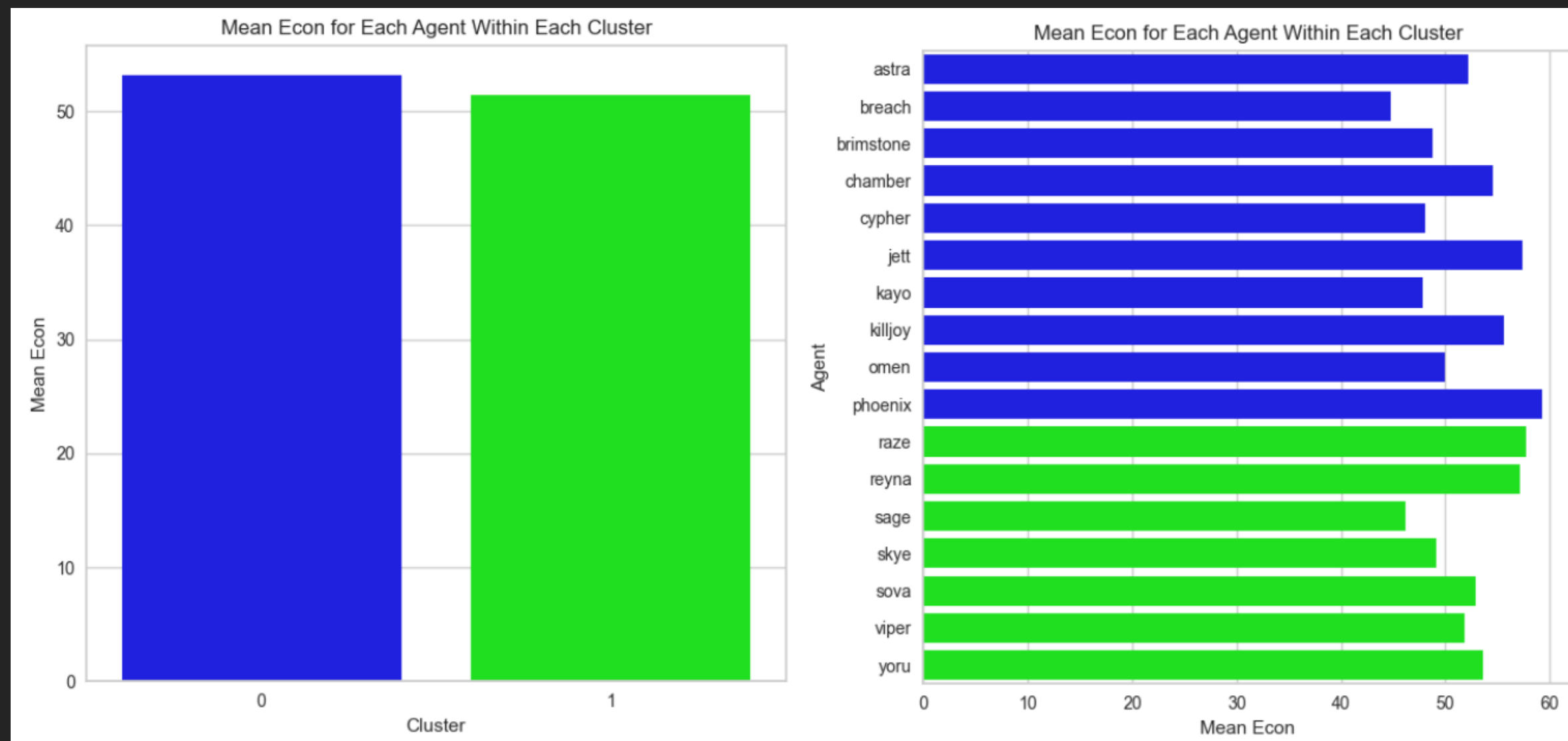
Interpretasi Cluster: Rata-rata FirstDeaths



Agent dengan rata-rata **FirstDeaths** tertinggi adalah Raze. Cluster 0 rata-rata **FirstDeath** sebesar 2.85 dan Cluster 1 rata-rata **FirstDeaths** sebesar 1.85

CLUSTERING #1

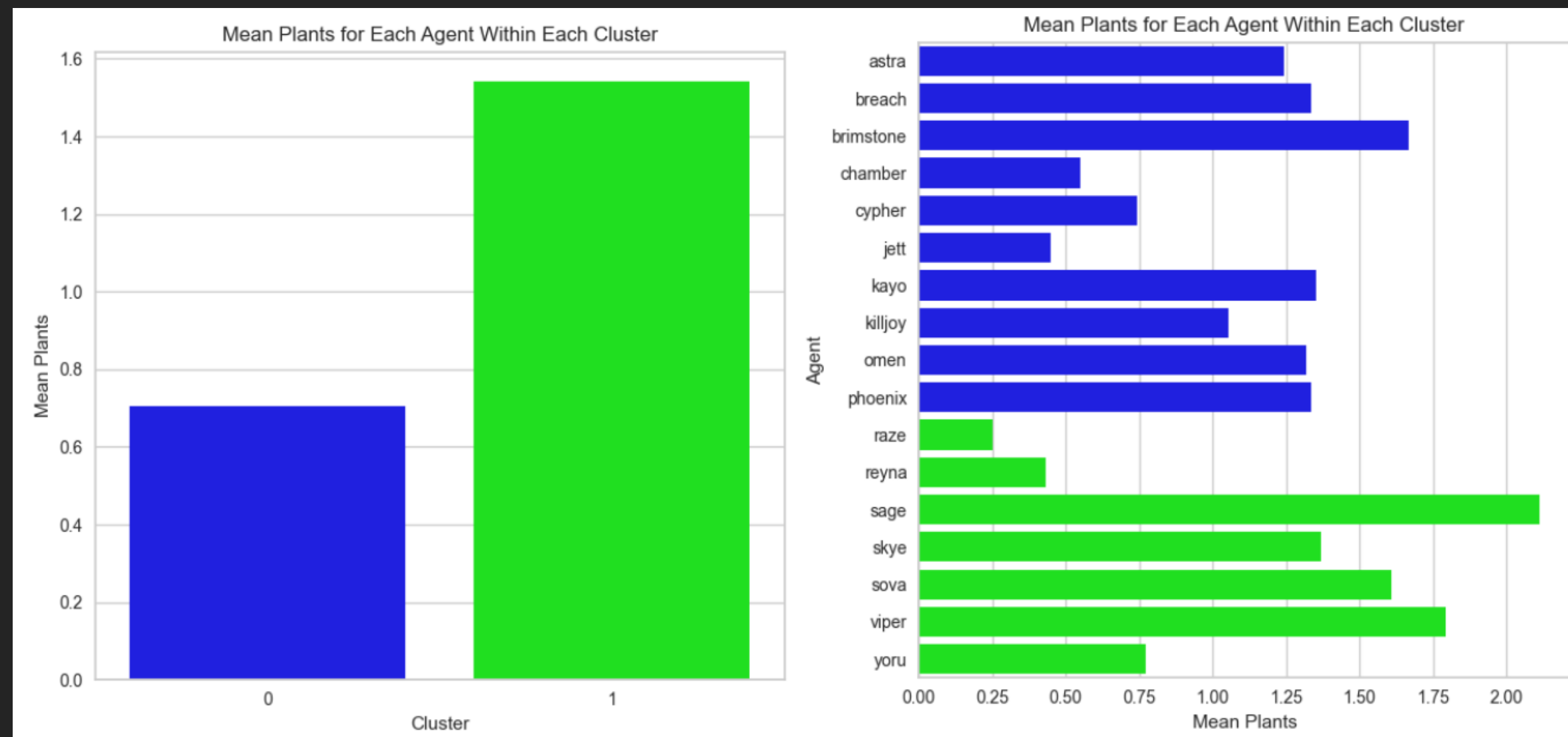
Interpretasi Cluster: Rata-rata Econ



Agent dengan rata-rata **Econ** tertinggi adalah Phoenix. Cluster 0 rata-rata **Econ** sebesar 55 dan Cluster 1 rata-rata **Econ** sebesar 52

CLUSTERING #1

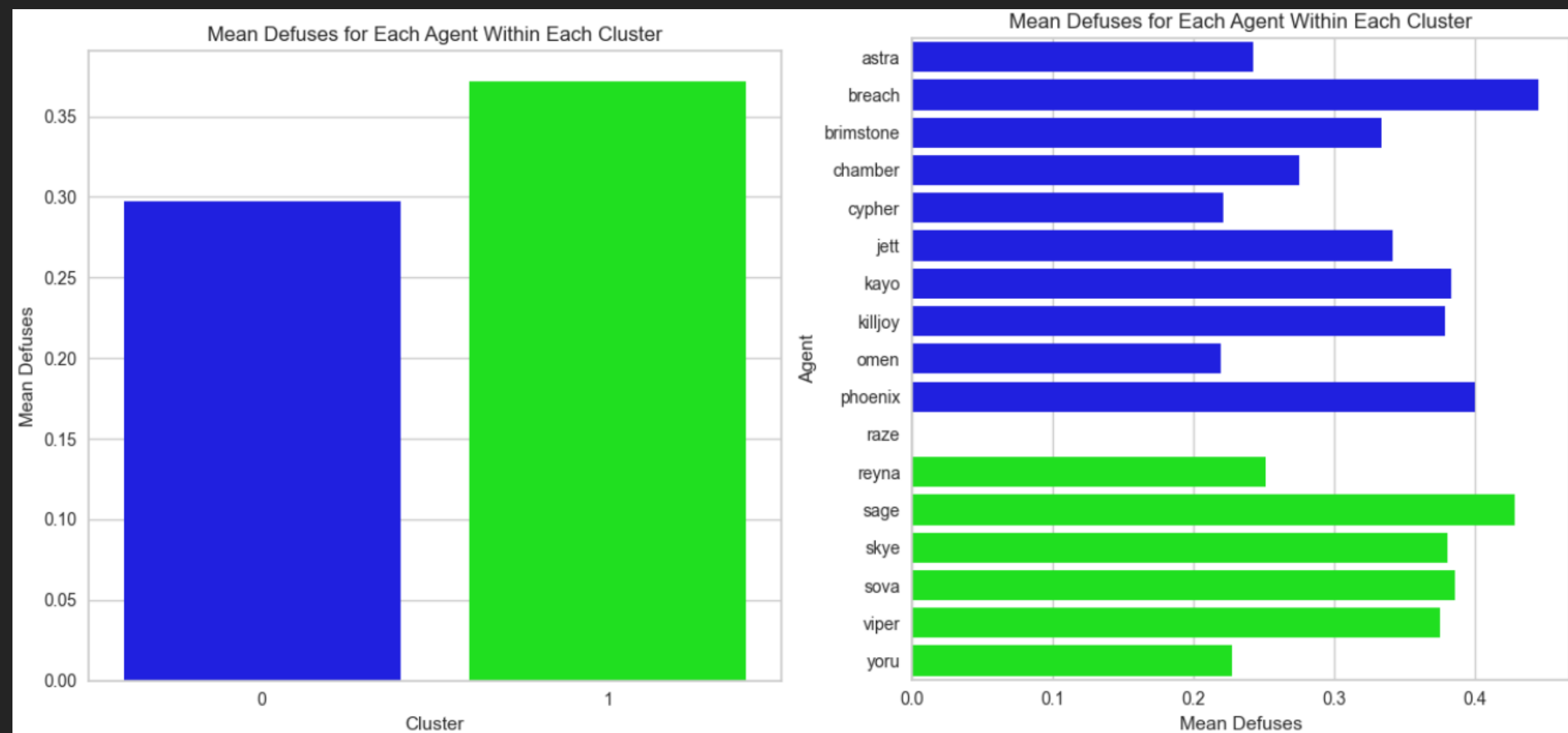
Interpretasi Cluster: Rata-rata Plants



Agent dengan rata-rata **Plants** tertinggi adalah Sage. Cluster 0 rata-rata **Plants** sebesar 0.7 dan Cluster 1 rata-rata **Plants** sebesar 1.5

CLUSTERING #1

Interpretasi Cluster: Rata-rata Defuses



Agent dengan rata-rata **Defuses** tertinggi adalah Breach. Cluster 0 rata-rata **Defuses** sebesar 0.29 dan Cluster 1 rata-rata **Defuses** sebesar 0.37

CLUSTERING #1 PENJELASAN

Cluster 0

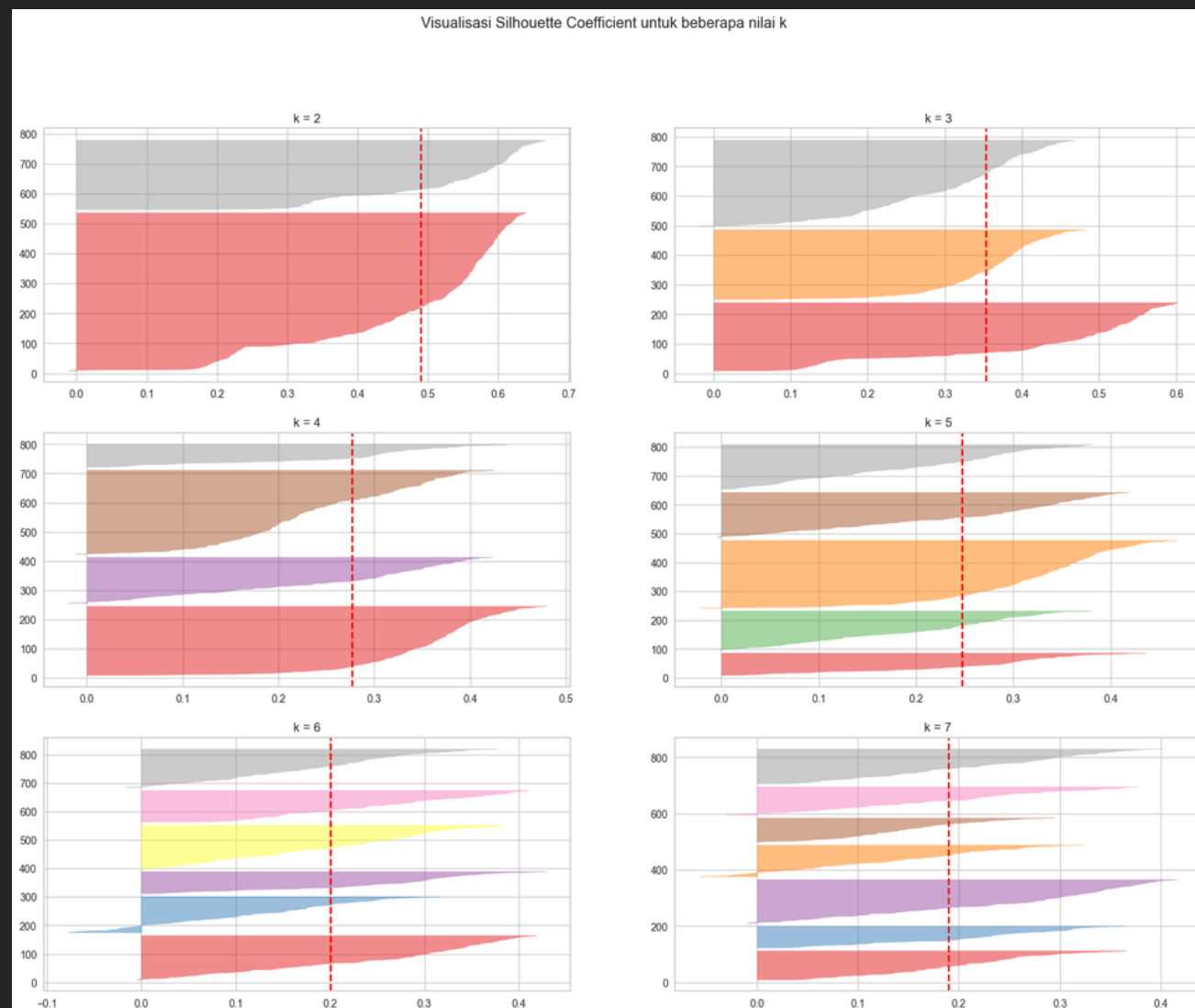
Memiliki nilai rata-rata **ACS** yang tinggi. **Agent** dalam cluster ini memiliki nilai rata-rata **Kills**, **Deaths**, **FirstKills**, dan **FirstDeaths** yang tinggi menandakan bahwa **Agent** dalam cluster ini lebih sering bermain dengan agresif sehingga mendapat banyak **Kills**, **Deaths**, **FirstKills**, dan **FirstDeaths**. Kemudian dari besaran serangan tersebut menyebabkan rata-rata **Econ** yang tinggi. Tidak hanya itu, **Agent** pada cluster ini jarang melakukan objektif yaitu **Plants** dan **Defuses**

Cluster 1

Memiliki nilai rata-rata **ACS** yang lebih rendah. **Agent** dalam cluster ini memiliki nilai rata-rata **Kills**, **Deaths**, **FirstKills**, dan **FirstDeaths** yang rendah menandakan bahwa **Agent** dalam cluster ini lebih sering bermain dengan pasif sehingga tidak mendapat banyak **Kills**, **Deaths**, **FirstKills**, dan **FirstDeaths**. Kemudian dari besaran serangan tersebut menyebabkan rata-rata **Econ** yang rendah. Tidak hanya itu, **Agent** pada cluster ini menjadi lebih sering melakukan objektif yaitu **Plants** dan **Defuses**

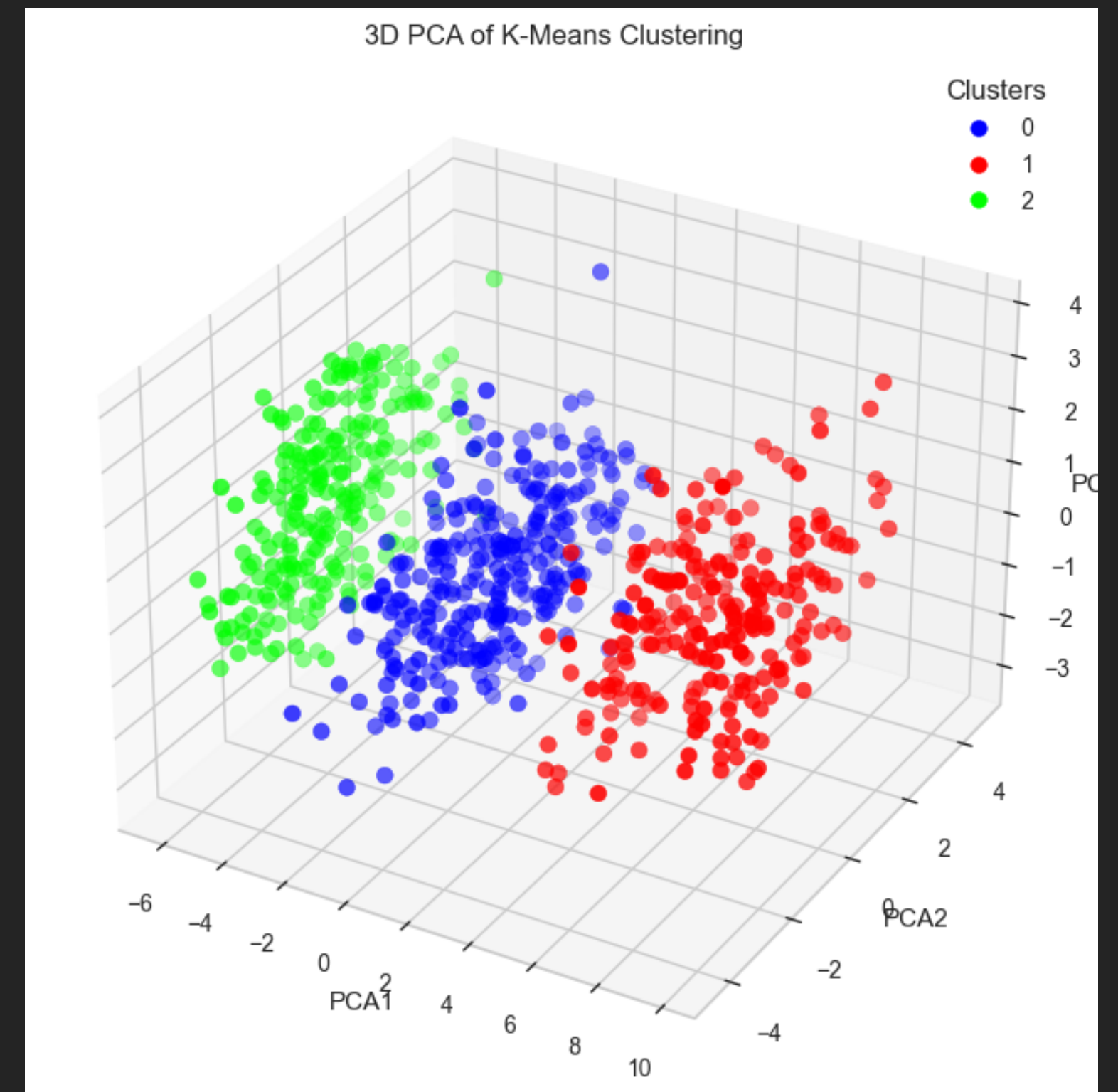
CLUSTERING #2

K-Means Clustering Map Fracture



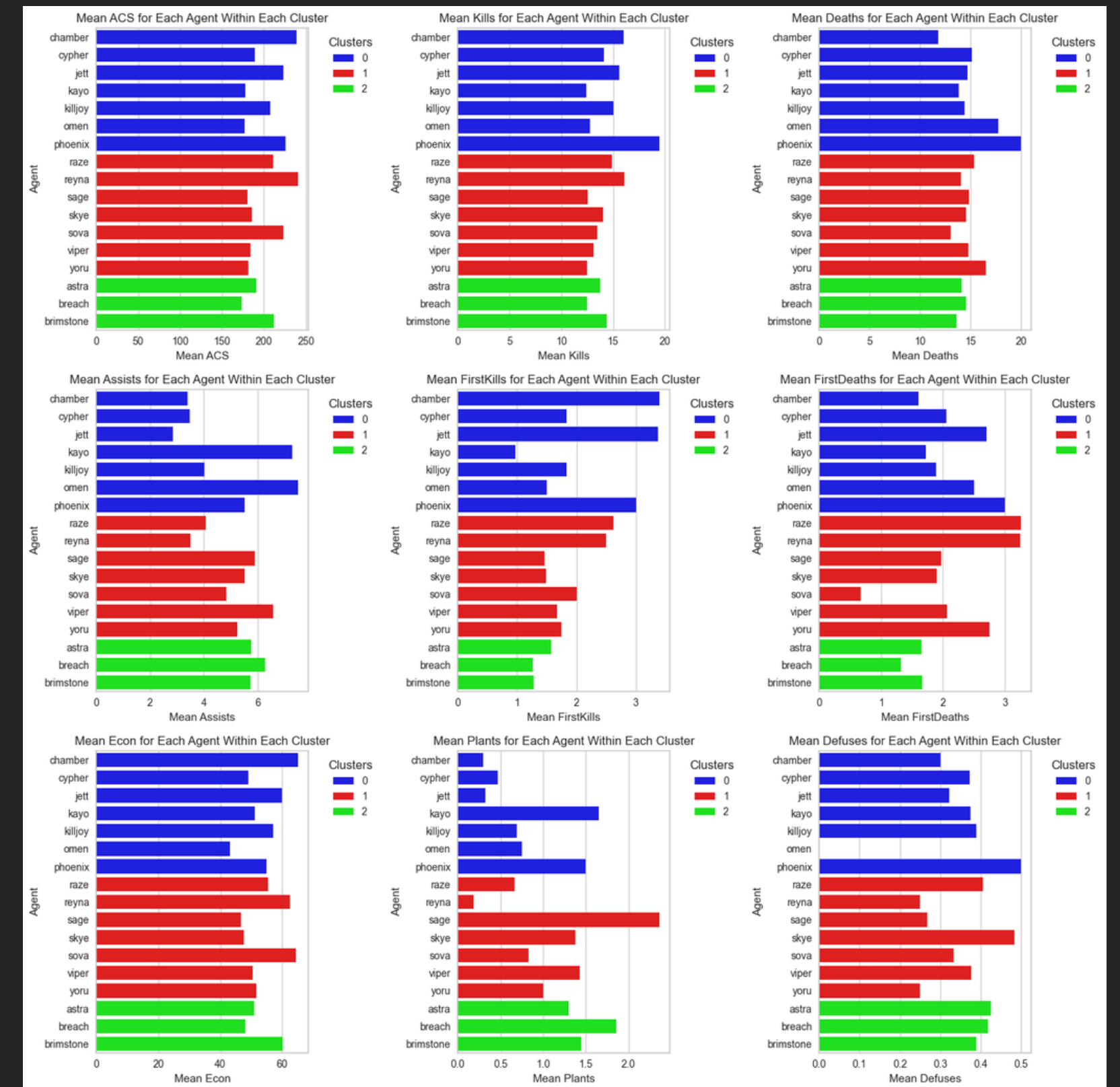
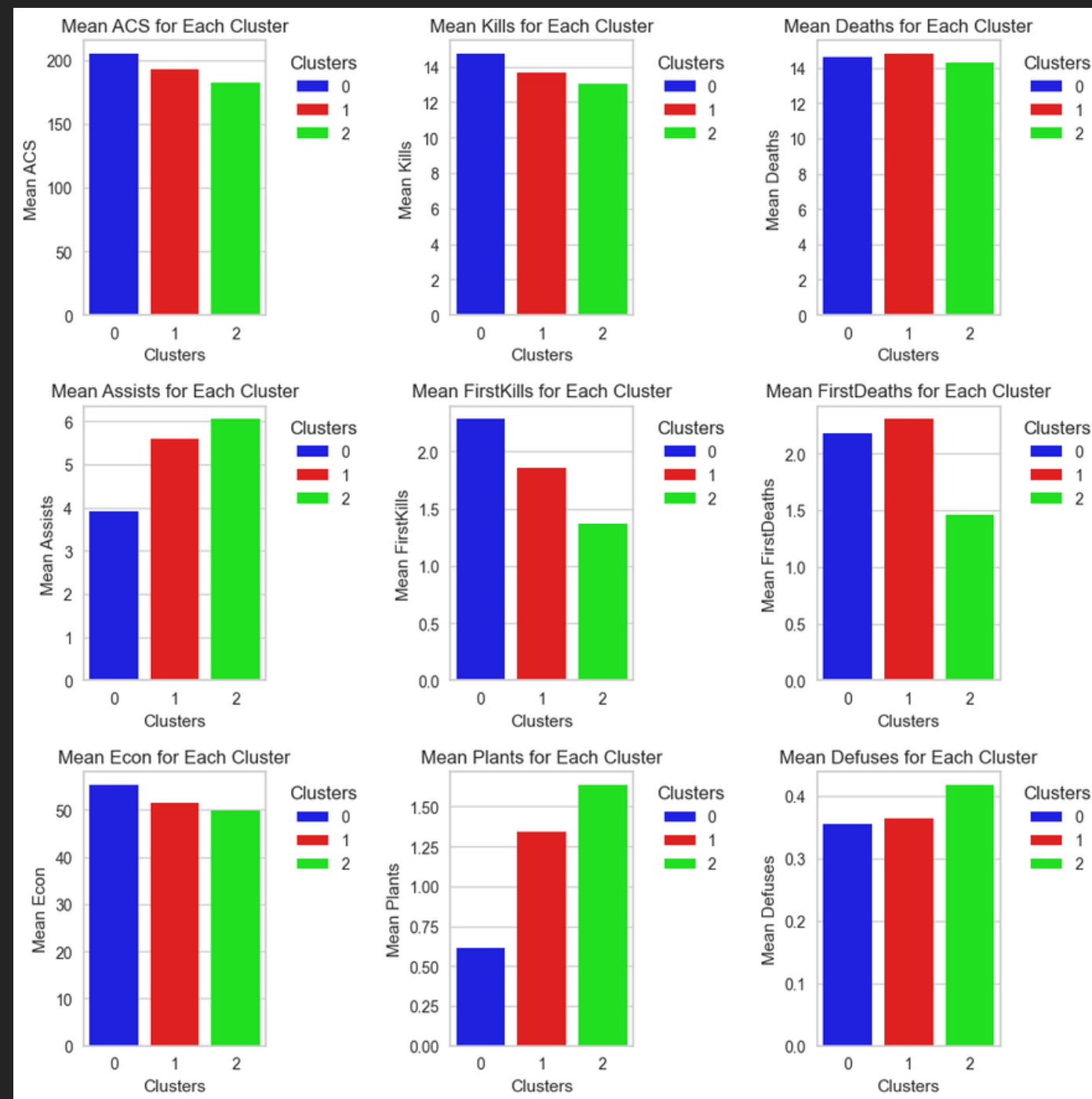
Silhouette Score: 0.35
Davies-Bouldin Index: 1.12
Calinski-Harabasz Index: 846.43

```
Clusters
0      293
2      237
1      231
Name: count, dtype: int64
```



CLUSTERING #2

K-Means Clustering Map Fracture



MODELING AND EVALUATION

CLASSIFICATION

Problem: Imbalanced Classification untuk memprediksi Agent yang digunakan oleh setiap pemain pada setiap babak

Feature Selection: Menghapus kolom-kolom yang tidak relevan

Feature Engineering: One Hot Encode kolom kategorikal, seperti kolom Map

Evaluasi: Train test split: 75% train dan 25% test + F1-Micro

Distribusi Kelas

Agent	
sova	15.274603
jett	14.786835
omen	9.968644
raze	8.388073
cypher	8.274204
killjoy	7.790685
sage	6.235607
astra	5.542195
viper	5.159799

breach	4.645689
skye	4.445143
reyna	4.156222
phoenix	2.692069
brimstone	1.631557
kayo	0.732501
yoru	0.141062
chamber	0.135113

Name: count, dtype: float64

RandomForestClassifier - 200 estimator + Cost Sensitive Learning

```
Accuracy Average: 0.3610469068660775
F1 Macro Average: 0.2392733818195874
F1 Micro Average: 0.36104690686607754
Precision Macro Average: 0.28746088516761475
Precision Micro Average: 0.3610469068660775
Recall Macro Average: 0.23314440848443432
Recall Micro Average: 0.3610469068660775
```

BaggingClassifier - 100 estimator

```
Accuracy Average: 0.35812372535690007
F1 Macro Average: 0.22322860561379668
F1 Micro Average: 0.35812372535690007
Precision Macro Average: 0.26139502411061166
Precision Micro Average: 0.35812372535690007
Recall Macro Average: 0.22297835796338225
Recall Micro Average: 0.35812372535690007
```

GradientBoostingClassifier - 500 estimator

```
Accuracy Average: 0.3991842284160435
F1 Macro Average: 0.2729780578976011
F1 Micro Average: 0.39918422841604356
Precision Macro Average: 0.31349607010322167
Precision Micro Average: 0.3991842284160435
Recall Macro Average: 0.2659041440760706
Recall Micro Average: 0.3991842284160435
```

HistGradientBoostingClassifier

```
Accuracy Average: 0.3863018354860639
F1 Macro Average: 0.2536748137226001
F1 Micro Average: 0.3863018354860639
Precision Macro Average: 0.29261672270592476
Precision Micro Average: 0.3863018354860639
Recall Macro Average: 0.25350005814298515
Recall Micro Average: 0.3863018354860639
```


REGRESSION

Problem: Memprediksi rata-rata nilai ACS untuk masing-masing tim yang bertanding

Feature Selection: Menghapus kolom-kolom yang tidak relevan

Evaluasi: Train test split: 80% train dan 20% test dan metrik R-squared

Random Forest Regressor

hyperparameter:

- `n_estimators = 100`
- `min_samples_split = 20`
- `criterion = poisson`

Hasil Evaluasi

```
Hasil Evaluasi Metrics untuk Random Forest Regressor
Mean Squared Error (MSE): 84.27604479066548
Mean Absoulute Error (MAE): 6.537468603399534
Root Mean Squared Error (RMSE): 9.180198515863667
R2 Score: 0.8935208259702739
```

Gradient Boosting Regressor

hyperparameter:

- `n_estimators = 500`
- `learning_rate = 0.1`
- `random_state = 42`
- `criterion = squared_error`

Hasil Evaluasi

```
Hasil Evaluasi Metrics untuk Gradient Boosting Regressor
Mean Squared Error (MSE): 78.61568133545592
Mean Absoulute Error (MAE): 6.412336860645391
Root Mean Squared Error (RMSE): 8.866548445446847
R2 Score: 0.9006724528283667
```

REGRESSION

Neural Networks (MLPRegressor)

Akan dilakukan standarisasi terlebih dahulu agar algoritma lebih cepat untuk konvergen

hyperparameter:

- **hidden_layer_sizes = (30,20)**
- **activation = relu**
- **alpha = 0.0001**
- **learning_rate = constant**
- **max_iter = 10000**

Hasil Evaluasi

```
Hasil Evaluasi Metrics untuk MLPRegressor
Mean Squared Error (MSE): 76.15389854316926
Mean Absoulute Error (MAE): 6.364064530926146
Root Mean Squared Error (RMSE): 8.726620109937711
R2 Score: 0.9037828099768825
```

Kesimpulan

Dari ketiga model yang dipilih dan diuji, model **MLPRegressor** dipilih untuk melakukan prediksi dikarenakan memiliki hasil evaluasi terbaik dengan nilai sebagai berikut.

- **MSE: 76.15389854316926**
- **MAE: 6.364064530926146**
- **RMSE: 8.726620109937711**
- **R-Squared: 0.9037828099768825**

Presented by

Data+

THANK YOU SO MUCH!

